

LAPORAN MILESTONE 2 TUGAS BESAR
IF2224 TEORI BAHASA FORMAL DAN OTOMATA
Syntax Analysis



Disusun oleh:

Juan Oloando Simanungkalit 13524032

Edward David Rumahorbo 13524036

Reinhard Alfonzo Hutabarat 13524056

Manuel Timothy Silalahi 13524102

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL.GANESA 10, BANDUNG 40132
2025

DAFTAR ISI

DAFTAR ISI.....	1
BAB 1	
LANDASAN TEORI.....	1
1. Syntax Analysis.....	1
2. Grammar.....	1
3. Parsing.....	1
3.1. Top-Down Parsing.....	2
3.2. Bottom-Up Parsing.....	2
4. Parse Tree.....	3
5. Recursive Descent Parsing.....	3
6. Left Recursion.....	4
BAB 2	
PERANCANGAN DAN IMPLEMENTASI.....	5
1. Perancangan.....	5
1.1. Arsitektur Umum.....	5
1.2. Struktur Program.....	5
Tabel 2.1 Struktur Program.....	5
2. Implementasi.....	7
2.1. Kakas yang Digunakan.....	7
2.2. Struktur Data ParseNode.....	7
2.3. Kelas Parser dan Fungsi Utility Inti.....	8
2.4. Parser_Core.cpp.....	10
2.5. Parser_Declarations.cpp.....	14
2.6. Parser_ControlFlow.cpp.....	20
2.7. Parser_Expression.cpp.....	26
2.8. Main.cpp.....	38
BAB 3	
PENGUJIAN.....	39
1. Test Case 1.....	39
2. Test Case 2.....	46
3. Test Case 3.....	51
4. Test Case 4.....	58
5. Test Case 5.....	65
BAB 4	
KESIMPULAN DAN SARAN.....	73
1. Kesimpulan.....	73

2. Saran.....	73
LAMPIRAN.....	75
REFERENSI.....	75

BAB 1

LANDASAN TEORI

1. Syntax Analysis

Syntax Analysis merupakan tahap dalam proses compiler yang bertugas untuk memeriksa apakah urutan token yang dihasilkan oleh *lexical analyzer* telah sesuai dengan aturan *grammar* bahasa pemrograman. Tahap ini sering disebut juga sebagai *parsing* karena proses utamanya adalah melakukan analisis struktur terhadap token-token *input* untuk menentukan apakah *source code* memiliki bentuk *syntax* yang valid. *Parser* akan menerima token-token dari *lexical analyzer* kemudian mencoba untuk menurunkannya berdasarkan *production rule* yang terdapat pada *grammar* hingga mencapai start symbol bahasa tersebut. Jika seluruh token dapat diturunkan dengan benar maka program dinyatakan valid secara *syntax*, sedangkan apabila tidak ditemukan turunan yang sesuai maka parser akan menghasilkan *syntax error*.

Syntax analysis memiliki peran penting dalam mendeteksi kesalahan *syntax* pada program serta memastikan struktur program sesuai dengan *grammar* yang ditentukan oleh bahasa pemrograman tertentu. Pada urutan *compiler*, *syntax analyzer* dilakukan setelah *lexical analyzer* dan sebelum *semantic analyzer*.

2. Grammar

Grammar merupakan kumpulan aturan formal yang digunakan untuk mendefinisikan struktur *syntax* suatu bahasa pemrograman. Dalam bidang *compiler*, *grammar* digunakan sebagai dasar dari parser untuk menentukan apakah suatu susunan token membentuk struktur bahasa yang valid atau tidak. *Grammar* mendeskripsikan bagaimana simbol-simbol dalam bahasa dapat dikombinasikan menjadi ekspresi, *statement*, maupun unit program yang lebih kompleks. Pada *syntax analysis*, jenis *grammar* yang paling umum digunakan adalah *Context-Free Grammar* (CFG). Secara formal, sebuah *Context-Free Grammar* dinyatakan sebagai:

$$G = (T, N, S, P)$$

dengan T sebagai himpunan terminal, N sebagai himpunan nonterminal, S sebagai *start symbol*, dan P sebagai kumpulan *production rule*.

3. Parsing

Parsing merupakan proses analisis *syntax* yang dilakukan parser terhadap urutan token hasil *lexical analysis* untuk menentukan apakah token-token tersebut dapat dibentuk

berdasarkan aturan *grammar* bahasa pemrograman. *Parsing* bekerja dengan mencoba melakukan derivasi (proses penurunan) dari *start symbol grammar* hingga menghasilkan urutan terminal yang sesuai dengan *input* program. Jika derivasi berhasil ditemukan, maka input dinyatakan valid secara *syntax*, sedangkan jika tidak maka parser akan memberikan informasi *syntax error*.

Dalam proses *parsing*, *parser* tidak hanya memvalidasi struktur *syntax*, tetapi juga membangun representasi program dalam bentuk *parse tree*. Secara umum terdapat dua pendekatan utama dalam parsing, yaitu *top-down parsing* dan *bottom-up parsing*.

3.1. Top-Down Parsing

Top-Down Parsing merupakan metode parsing yang membangun *parse tree* dimulai dari *root* menuju *leaf*. Parser memulai proses dari *start symbol grammar* kemudian secara bertahap menurunkan nonterminal menjadi simbol-simbol yang lebih spesifik hingga seluruh input berhasil dicocokkan. Pendekatan ini mengikuti urutan *leftmost derivation* karena *parser* selalu mengembangkan nonterminal paling kiri terlebih dahulu.

Pada *top-down parsing*, *parser* harus menentukan *production rule* mana yang akan digunakan berdasarkan token *input* saat ini. Oleh karena itu, metode ini biasanya memanfaatkan teknik *lookahead* untuk melihat token-token berikutnya sebelum *parsing*. Salah satu implementasi paling umum dari metode ini adalah *recursive descent*, yaitu parser yang dibangun menggunakan fungsi rekursif untuk setiap nonterminal *grammar*.

3.2. Bottom-Up Parsing

Bottom-Up parsing merupakan metode parsing yang bekerja dengan membangun *parse tree* dari *leaf* menuju *root*. Pada pendekatan ini *parser* memulai proses dari token-token *input* kemudian secara bertahap melakukan reduksi terhadap *substring* tertentu hingga akhirnya menghasilkan *start symbol*.

Bottom-up parsing umumnya diimplementasikan menggunakan teknik *shift-reduce parsing*. Parser akan menyimpan simbol-simbol *input* pada *stack*, kemudian melakukan operasi *shift* untuk memasukkan token baru dan *reduce* untuk mengganti sekumpulan simbol dengan nonterminal tertentu sesuai *production rule grammar*. Selain *shift* dan *reduce*, *parser* juga memiliki aksi *accept* ketika *parsing* berhasil serta *error* ketika ditemukan kesalahan *syntax*.

4. *Parse Tree*

Parse Tree adalah struktur data berbentuk *tree* yang merepresentasikan derivasi lengkap dari suatu input berdasarkan *grammar*. Setiap node internal dalam *parse tree* merepresentasikan sebuah non-terminal (seperti `<statement>` atau `<expression>`), sedangkan setiap *leaf node* merepresentasikan sebuah terminal atau token aktual dari input (seperti `IDENT(a)` atau `BECOMES`).

Parse Tree berbeda dari Abstract Syntax Tree (AST). *Parse Tree* mempertahankan semua simbol *grammar* termasuk yang bersifat struktural (seperti tanda kurung dan semicolon), sehingga pohonnya lebih *verbose* namun lebih lengkap secara struktural. AST sebaliknya membuang simpul-simpul yang tidak memiliki nilai semantik dan hanya mempertahankan informasi yang esensial untuk evaluasi program. Keluaran yang diharapkan pada *milestone* ini adalah *Parse Tree* penuh, bukan AST.

Dalam implementasi proyek ini, setiap node *parse tree* direpresentasikan oleh kelas `ParseNode` yang memiliki atribut `name` (berupa *string* seperti `"<statement>"` untuk non-terminal atau `"IDENT>Hello)"` untuk *leaf node*) dan `children` (berupa `std::vector<ParseNode*>` yang menampung node-node anak).

5. *Recursive Descent Parsing*

Recursive Descent Parser adalah implementasi *top-down parser* di mana setiap non-terminal dalam *grammar* dipetakan secara langsung ke sebuah fungsi rekursif dalam kode program. Ketika parser perlu memproses suatu non-terminal, ia memanggil fungsi yang bersesuaian. Fungsi tersebut kemudian memeriksa token saat ini (*lookahead*) untuk menentukan aturan produksi mana yang harus diikuti, lalu memanggil fungsi-fungsi lain secara rekursif untuk menangani non-terminal yang ada di sisi kanan aturan produksi tersebut.

Mekanisme kunci dalam *Recursive Descent* adalah fungsi `match()` dan `peekToken()`. Fungsi `peekToken(int offset = 0)` digunakan untuk melihat token saat ini atau token berikutnya tanpa mengolahnya (*lookahead*), sedangkan fungsi `match(TokenType expectedType)` digunakan untuk mencocokkan dan mengolah token saat ini. Jika token yang ada sesuai dengan yang diharapkan, token dikonsumsi dan sebuah *leaf node* dibuat. Jika tidak sesuai, mekanisme error handling akan dipicu.

Recursive Descent bekerja dengan baik untuk *grammar* LL(1), yaitu *grammar* yang dapat menentukan aturan produksi mana yang harus digunakan hanya dengan melihat satu

token ke depan (1 token lookahead). Grammar bahasa Arion dirancang untuk memenuhi properti ini pada sebagian besar produksinya.

6. *Left Recursion*

Left recursion adalah kondisi di mana sebuah non-terminal A dapat diturunkan ke suatu rangkaian yang dimulai dengan A itu sendiri, baik secara langsung ($A \rightarrow A\alpha$) maupun tidak langsung. *Left recursion* merupakan masalah serius bagi *Recursive Descent Parser* karena akan menyebabkan infinite recursion, di mana fungsi untuk A akan memanggil dirinya sendiri tanpa batas tanpa pernah mengolah token apapun, yang berujung pada *stack overflow*.

Sebagai contoh, grammar *left-recursive* untuk ekspresi seperti $\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$ tidak dapat langsung diimplementasikan sebagai fungsi rekursif. Solusinya adalah dengan melakukan transformasi grammar menjadi bentuk yang ekuivalen namun tidak *left-recursive*, umumnya dengan menggantikan rekursi kiri menjadi perulangan (iterasi) menggunakan simbol * dalam EBNF.

Pada Milestone ini, grammar bahasa Arion sudah dirancang dalam bentuk EBNF yang bebas dari *left recursion*. Contohnya, aturan $\langle \text{simple-expression} \rangle \rightarrow (\text{plus} \mid \text{minus})? \text{ term } (\text{additive-operator } \text{term})^*$ menggunakan simbol * untuk menyatakan pengulangan, yang ditranslasikan langsung menjadi perulangan *while* dalam implementasi, bukan rekursi kiri. Hal yang sama berlaku untuk $\langle \text{term} \rangle$, $\langle \text{statement-list} \rangle$, $\langle \text{identifier-list} \rangle$, dan seluruh aturan lain yang bersifat berulang.

BAB 2

PERANCANGAN DAN IMPLEMENTASI

1. Perancangan

1.1. Arsitektur Umum

Pada Milestone 2 Arion Compiler, program dibangun dengan arsitektur *pipeline* dua tahap yang mengintegrasikan komponen Lexer dari Milestone 1 dengan komponen *Parser* yang baru dikembangkan. Secara umum, alur kerja program adalah sebagai berikut: program menerima *source code* Arion dari sebuah file teks, kemudian Lexer akan mengubah kode tersebut menjadi deretan token. Token-token tersebut selanjutnya diambil alih oleh *Parser* untuk membangun *Parse Tree* sesuai dengan *grammar* bahasa Arion. Hasil *Parse Tree* kemudian dicetak ke terminal dan disimpan ke dalam file output.

Arsitektur *pipeline* ini memisahkan tanggung jawab antara analisis *lexical* dan analisis *syntax*. Lexer bertanggung jawab mengidentifikasi unit *lexical*(token), sedangkan *Parser* bertanggung jawab memeriksa apakah urutan token tersebut membentuk struktur yang valid sesuai *grammar* bahasa Arion. Pendekatan ini memudahkan dalam pengembangan dan pengujian masing-masing komponen secara independen.

1.2. Struktur Program

Program dikembangkan menggunakan bahasa C++ dan dibagi ke dalam beberapa file *source* (.cpp) dan *header* (.hpp) dimana masing-masing punya tanggung jawab tersendiri. Berikut adalah daftar file dalam folder src/ beserta tanggung jawabnya:

Tabel 2.1 Struktur Program

File	Tanggung Jawab
Token.hpp / Token.cpp	Definsi struktur Token dan TokenType, serta fungsi toString() untuk representasi dari token.
Lexer.hpp / Lexer.cpp	Kelas Lexer untuk membaca <i>source code</i> karakter per karakter dan menghasilkan token dengan bantuan Lexer_Keywords.cpp, Lexer_Literals.cpp, Lexer_Operators.cpp, dan Lexer_DelimsComments.cpp.
ParseNode.hpp / ParseNode.cpp	Definisi struktur node pada Parse

	Tree, termasuk fungsi addChild() dan print() untuk mencetak pohon ke output.
Parser.hpp	Deklarasi kelas Parser beserta seluruh fungsi parse yang mencakup grammar non-terminal grammar bahasa Arion.
Parser_Controlflow.hpp / Parser_Controlflow.cpp	Implementasi fungsi untuk parsing subprogram dan control flow. Fungsi tersebut adalah parseProcedureDeclaration(), parseFunctionDeclaration(), parseFormalParameterList(), parseIfStatement(), parseCaseStatement(), parseWhileStatement(), parseRepeatStatement(), dan parseForStatement().
Parser.cpp	Implementasi konstruktor Parser, fungsi parse(), serta fungsi-fungsi inti, seperti peekToken(), advToken(), reportError(), dan match().
Parser_Core.hpp / Parser_Core.cpp	Implementasi fungsi untuk struktur program inti, yaitu parseProgram(), parseProgramHeader(), parseDeclarationPart(), parseBlock(), parseCompoundStatement(), parseStatement(), dan parseStatementList().
Parser_Declarations.hpp / Parser_Declarations.cpp	Implementasi fungsi untuk parsing deklarasi, yaitu parseConstDeclaration(), parseTypeDeclaration(), parseVarDeclaration(), parseType(), parseArrayType(), parseRange(), parseEnumerated(), parseRecordType(), parseFieldList(), dan parseFieldPart().
Parser_Expressions.hpp / Parser_Expressions.cpp	Implementasi fungsi untuk parsing ekspresi dan pernyataan, yaitu

	parseAssignmentStatement(), parseProcedureFunctionCall(), parseExpression(), parseSimpleExpression(), parseTerm(), parseFactor(), parseVariable(), parseComponentVariable(), dan parseIndexList().
Main.cpp	Entry point program utama. Membaca argumen command-line, membuat instance untuk Lexer dan parser, memanggil proses parsing, serta mengeluarkan hasil output ke file dan terminal.

2. Implementasi

2.1. Kakas yang Digunakan

Implementasi program menggunakan kakas sebagai berikut:

- Bahasa Pemrograman: C++, alasan pemilihan adalah karena kemampuan manajemen memori manual yang baik untuk membangun struktur pohon (*tree*), serta adanya *standar library* yang memadai seperti `std::vector`, `std::string`, dan lain sebagainya.
- Compiler: g++ (GNU Compiler Collection) yang mendukung standar C++17.

2.2. Struktur Data ParseNode

Setiap simpul pada *Parse Tree* direpresentasikan oleh kelas ParseNode yang diimplementasikan pada ParseNode.hpp. Kelas ini memiliki fungsionalitas utama untuk membangun struktur pohon:

```
class ParseNode {
public:
    std::string name;
    std::vector<ParseNode*> children;

    ParseNode(const std::string& name);
    ~ParseNode();
    void addChild(ParseNode* child);
    void printTree(std::ostream& out, const
std::string& prefix = "",
```

```
bool isLast = true, bool isRoot =  
true) const;  
};
```

Atribut name menyimpan label node sebagai string. Untuk node non-terminal (seperti node hasil parse production rule), name diisi dengan format <nama-nonterminal>, misalnya “<program>” atau “<statement>”. Untuk node terminal (leaf node), name diisi dengan representasi token yang sesuai, misal “IDENT(Hello)” atau “SEMICOLON”.

Atribut children adalah vektor pointer ke ParseNode, yang menyimpan semua simpul anak secara berurutan sesuai urutan token yang diolah. Destruktor kelas ini secara rekursif menghapus seluruh anak, sehingga cukup memanggil delete pada node root untuk membebaskan seluruh memori pohon.

Fungsi printTree() mencetak pohon ke output stream dengan format visual yang hierarkis. Fungsi ini juga dipanggil secara rekursif pada setiap anak.

2.3. Kelas Parser dan Fungsi *Utility* Inti

Kelas Parser didefinisikan dalam Parser.hpp dan diimplementasikan dalam beberapa file .cpp. Kelas ini memiliki tiga atribut privat:

```
class Parser {  
private:  
    std::vector<Token> tokens;  
    size_t currTokenIdx;  
    bool hasError;
```

- tokens: std::vector<Token> yang menyimpan seluruh token hasil tokenisasi dari Lexer. Token bertipe COMMENT dibuang, sementara token bertipe UNKNOWN dilaporkan sebagai Lexical Error ke stderr.
- currTokenIdx: size_t yang menjadi penanda posisi token yang sedang diproses saat ini
- hasError: bool yang menjadi flag apakah telah terjadi syntax error selama proses parsing atau tidak.

Konstruktor Parser menerima referensi ke objek Lexer, memanggil lexer.tokenizeAll() untuk memperoleh seluruh token sekaligus, kemudian

memfilter token yang tidak relevan (COMMENT dibuang, UNKNOWN diberikan ke stderr untuk ditangani).

Empat fungsi utility yang menjadi fondasi seluruh proses parsing adalah:

```
/ Melihat token ke-offset dari posisi saat ini tanpa
mengolahnya
Token Parser::peekToken(int offset) const {
    if (currTokenIdx + offset < tokens.size())
        return tokens[currTokenIdx + offset];
    return tokens.back(); // EOF guard
}

// Maju ke token berikutnya
void Parser::advToken() {
    if (currTokenIdx < tokens.size() - 1)
        currTokenIdx++;
}

// Melaporkan syntax error (hanya error pertama yang
dilaporkan)
void Parser::reportError(const std::string& message)
{
    if (!hasError) {
        std::cerr << "Syntax Error at Line " <<
peekToken().line
                << ", Col " << peekToken().column
<< ": " << message << "\n";
        hasError = true;
    }
}

// Mencocokkan dan menangani token dengan tipe yang
diharapkan
ParseNode* Parser::match(TokenType expectedType) {
    Token curr = peekToken();
    if (curr.type == expectedType) {
        ParseNode* node = new
```

```

ParseNode(curr.toString());
    advToken();
    return node;
} else {
    reportError("Unexpected token " +
curr.toString());
    return new ParseNode("ERROR");
}
}

```

Fungsi `peekToken(int offset = 0)` memungkinkan lookahead lebih dari satu token ke depan, yang diperlukan untuk menentukan production rule mana yang harus digunakan. Sebagai contoh, pada `parseStatement()`, pemanggilan `peekToken(1)` digunakan untuk membedakan antara assignment-statement (diawali IDENT diikuti BECOMES) dengan procedure/function-call (diawali IDENT diikuti selain BECOMES).

Fungsi `match()` adalah fungsi terminal yang mengolah token dan menciptakan leaf node. Jika token yang ada tidak sesuai dengan yang diharapkan, fungsi ini memanggil `reportError()` dan mengembalikan sebuah node ERROR sebagai bentuk graceful error handling agar parsing dapat terus dilanjutkan semaksimal mungkin.

2.4. Parser_Core.cpp

File ini mengimplementasikan fungsi-fungsi yang berhubungan dengan struktur inti program Arion.

parseProgram()

```

ParseNode* Parser::parseProgram() {
    ParseNode* node = new ParseNode("<program>");

    node->addChild(parseProgramHeader());
    node->addChild(parseDeclarationPart());
    node->addChild(parseCompoundStatement());
    node->addChild(match(TokenType::PERIOD));

    return node;
}

```

```
}
```

Fungsi ini merupakan titik masuk proses parsing, memetakan production rule program $\rightarrow$ program-header + declaration-part + compound-statement + period. Fungsi ini memanggil secara berurutan `parseProgramHeader()`, `parseDeclarationPart()`, `parseCompoundStatement()`, dan `match(PERIOD)`.

parseProgramHeader()

```
ParseNode* Parser::parseProgramHeader() {  
    ParseNode* node = new  
    ParseNode("<program-header>");  
  
    node→addChild(match(TokenType::PROGRAMSY));  
    node→addChild(match(TokenType::IDENT));  
    node→addChild(match(TokenType::SEMICOLON));  
  
    return node;  
}
```

Memetakan production rule program-header $\rightarrow$ programsy + ident + semicolon. Fungsi ini memanggil `match()` tiga kali secara berurutan untuk token PROGRAMSY, IDENT, dan SEMICOLON.

parseDeclarationPart()

```
ParseNode* Parser::parseDeclarationPart(){  
    ParseNode* node = new  
    ParseNode("<declaration-part>");  
  
    while (peekToken().type == TokenType::CONSTSY) {  
        node→addChild(parseConstDeclaration());  
    }  
    while (peekToken().type == TokenType::TYPESY) {  
        node→addChild(parseTypeDeclaration());  
    }  
    while (peekToken().type == TokenType::VARSY) {  
        node→addChild(parseVarDeclaration());  
    }  
    while (peekToken().type == TokenType::PROCEDURESY
```

```

|| peekToken().type == TokenType::FUNCTIONSY) {
    node→addChild(parseSubprogramDeclaration());
}

return node;
}

```

Memetakan production rule declaration-part -> (const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*. Setiap bagian diproses dalam loop while yang memeriksa token lookahead saat ini. Urutan pemrosesan bersifat tetap: konstan -> tipe -> variabel -> subprogram. Setiap jenis deklarasi dapat muncul nol atau lebih kali.

parseBlock()

```

ParseNode* Parser::parseBlock(){
    ParseNode* node = new ParseNode("<block>");

    node→addChild(parseDeclarationPart());
    node→addChild(parseCompoundStatement());

    return node;
}

```

Memetakan production rule block -> declaration-part + compound-statement. Digunakan oleh deklarasi prosedur dan fungsi untuk merepresentasikan tubuh (body) subprogram.

parseCompoundStatement()

```

ParseNode* Parser::parseCompoundStatement(){
    ParseNode* node = new
    ParseNode("<compound-statement>");

    node→addChild(match(TokenType::BEGINSY));
    node→addChild(parseStatementList());
    node→addChild(match(TokenType::ENDSY));

    return node;
}

```

Memetakan production rule compound-statement -> beginsy + statement-list + endsy. Fungsi ini mengolah token BEGINSY, memanggil parseStatementList(), kemudian mengolah token ENDSY.

parseStatement()

```
ParseNode* Parser::parseStatement(){
    ParseNode* node = new ParseNode("<statement>");
    TokenType type = peekToken().type;

    if (type == TokenType::IDENT) {
        if (peekToken(1).type == TokenType::BECOMES)
        {
            node→addChild(parseAssignmentStatement());
        } else {
            node→addChild(parseProcedureFunctionCall());
        }
    } else if (type == TokenType::IFSY) {
        node→addChild(parseIfStatement());
    } else if (type == TokenType::CASESY) {
        node→addChild(parseCaseStatement());
    } else if (type == TokenType::WHILESY) {
        node→addChild(parseWhileStatement());
    } else if (type == TokenType::REPEATSY) {
        node→addChild(parseRepeatStatement());
    } else if (type == TokenType::FORSY) {
        node→addChild(parseForStatement());
    }

    return node;
}
```

Memetakan production rule statement -> (assignment-statement | if-statement | case-statement | while-statement | repeat-statement | for-statement | procedure/function-call)?. Fungsi ini menggunakan token lookahead untuk menentukan jenis statement yang akan diparsing. Jika token saat ini adalah IDENT, dilakukan lookahead tambahan pada token berikutnya: jika BECOMES maka dispatch ke parseAssignmentStatement(), jika tidak maka dispatch ke

parseProcedureFunctionCall(). Statement dapat pula kosong (epsilon production), dalam hal ini node <statement> dikembalikan tanpa anak.

parseStatementList()

```
ParseNode* Parser::parseStatementList(){
    ParseNode* node = new
    ParseNode("<statement-list>");

    node→addChild(parseStatement());

    while (peekToken().type == TokenType::SEMICOLON)
    {
        node→addChild(match(TokenType::SEMICOLON));
        node→addChild(parseStatement());
    }

    return node;
}
```

Memetakan production rule statement-list -> statement (semicolon + statement)*. Fungsi ini memanggil parseStatement() satu kali, kemudian terus mengolah SEMICOLON dan memanggil parseStatement() selama token saat ini adalah SEMICOLON.

2.5. Parser_Declarations.cpp

File ini mengimplementasikan seluruh fungsi yang berkaitan dengan parsing deklarasi variabel, konstanta, tipe data, dan subprogram.

parseConstDeclaration()

```
ParseNode* Parser::parseConstDeclaration() {
    ParseNode* node = new
    ParseNode("<const-declaration>");

    node→addChild(match(TokenType::CONSTSY));

    while (peekToken().type == TokenType::IDENT) {
        node→addChild(match(TokenType::IDENT));
        node→addChild(match(TokenType::EQL));
    }
}
```

```

        node→addChild(parseConstant());
        node→addChild(match(TokenType::SEMICOLON));
    }

    return node;
}

```

Memetakan production rule const-declaration $\rightarrow$ constsy + (ident + eql + constant + semicolon)+. Fungsi ini mengonsumsi token CONSTSY, kemudian dalam loop while selama token saat ini IDENT, secara berurutan mengonsumsi IDENT, EQL, memanggil parseConstant(), dan mengonsumsi SEMICOLON.

parseConstant()

```

ParseNode* Parser::parseConstant() {
    ParseNode* node = new ParseNode("<constant>");

    TokenType type = peekToken().type;

    if (type == TokenType::CHARCON || type ==
        TokenType::STRING) {
        node→addChild(match(type));
    }
    else {
        if (type == TokenType::PLUS || type ==
            TokenType::MINUS) {
            node→addChild(match(type));
            type = peekToken().type;
        }

        if (type == TokenType::IDENT ||
            type == TokenType::INTCON ||
            type == TokenType::REALCON) {
            node→addChild(match(type));
        }
        else {
            reportError("Expected constant");
            node→addChild(new ParseNode("ERROR"));
            advToken();
        }
    }
}

```

```

        }
    }

    return node;
}

```

Memetakan production rule constant -> charcon | string | [(plus | minus)? + (ident | intcon | realcon)]. Fungsi helper isConstantStart() digunakan untuk memeriksa apakah suatu TokenType dapat mengawali sebuah konstanta.

parseTypeDeclaration() dan parseVarDeclaration()

```

ParseNode* Parser::parseTypeDeclaration() {
    ParseNode* node = new
    ParseNode("<type-declaration>");

    node->addChild(match(TokenType::TYPESY));

    while (peekToken().type == TokenType::IDENT) {
        node->addChild(match(TokenType::IDENT));
        node->addChild(match(TokenType::EQL));
        node->addChild(parseType());
        node->addChild(match(TokenType::SEMICOLON));
    }

    return node;
}

ParseNode* Parser::parseVarDeclaration() {
    ParseNode* node = new
    ParseNode("<var-declaration>");

    node->addChild(match(TokenType::VARSY));

    while (peekToken().type == TokenType::IDENT) {
        node->addChild(parseIdentifierList());
        node->addChild(match(TokenType::COLON));
        node->addChild(parseType());
        node->addChild(match(TokenType::SEMICOLON));
    }
}

```

```

    }

    return node;
}

```

Keduanya mengikuti pola yang sama dengan `parseConstDeclaration()`, namun untuk deklarasi tipe dan variabel masing-masing. `parseVarDeclaration()` menggunakan `parseIdentifierList()` untuk mendukung deklarasi beberapa variabel sekaligus dalam satu baris (misalnya `var a, b, c: integer;`).

parseType()

```

ParseNode* Parser::parseType() {
    ParseNode* node = new ParseNode("<type>");

    TokenType type = peekToken().type;

    if (type == TokenType::ARRAYSY) {
        node→addChild(parseArrayType());
    }
    else if (type == TokenType::RECORDSY) {
        node→addChild(parseRecordType());
    }
    else if (type == TokenType::LPARENT) {
        node→addChild(parseEnumerated());
    }
    else if (type == TokenType::IDENT) {
        if (peekToken(1).type == TokenType::PERIOD &&
            peekToken(2).type == TokenType::PERIOD) {
            node→addChild(parseRange());
        }
        else {
            node→addChild(match(TokenType::IDENT));
        }
    }
    else if (isConstantStart(type)) {
        node→addChild(parseRange());
    }
    else {

```

```

        reportError("Expected type");
        node→addChild(new ParseNode("ERROR"));
        advToken();
    }

    return node;
}

```

Fungsi ini bertanggung jawab menentukan tipe data berdasarkan token lookahead.

Logika dispatching-nya adalah:

- Jika token saat ini ARRAYSY -> parseArrayType()
- Jika token saat ini RECORDSY -> parseRecordType()
- Jika token saat ini LPARENT -> parseEnumerated()
- Jika token saat ini IDENT dan diikuti dua PERIOD berurutan -> parseRange() (subrange type)
- Jika token saat ini IDENT saja -> match(IDENT) (tipe named)
- Jika token dapat mengawali konstanta (bukan IDENT) -> parseRange()

parseArrayType()

```

ParseNode* Parser::parseArrayType() {
    ParseNode* node = new ParseNode("<array-type>");

    node→addChild(match(TokenType::ARRAYSY));
    node→addChild(match(TokenType::LBRACK));

    if (peekToken().type == TokenType::IDENT &&
        peekToken(1).type != TokenType::PERIOD) {
        node→addChild(match(TokenType::IDENT));
    }
    else {
        node→addChild(parseRange());
    }

    node→addChild(match(TokenType::RBRACK));
    node→addChild(match(TokenType::OFSY));
    node→addChild(parseType());

    return node;
}

```

```
}
```

Memetakan production rule array-type -> arraysy + lbrack + (range | ident) + rbrack + ofsy + type. Untuk menentukan apakah index array berupa ident atau range, dilakukan lookahead: jika token saat ini IDENT dan token berikutnya bukan PERIOD, maka langsung dikonsumsi sebagai ident; jika tidak, diparsing sebagai range.

parseRange()

```
ParseNode* Parser::parseRange() {  
    ParseNode* node = new ParseNode("<range>");  
  
    node→addChild(parseConstant());  
    node→addChild(match(TokenType::PERIOD));  
    node→addChild(match(TokenType::PERIOD));  
    node→addChild(parseConstant());  
  
    return node;  
}
```

Memetakan production rule range -> constant + period + period + constant. Dua token PERIOD dikonsumsi secara berurutan untuk merepresentasikan simbol ".." yang digunakan sebagai pemisah batas bawah dan atas range.

parseRecordType() dan parseFieldList()

```
ParseNode* Parser::parseRecordType() {  
    ParseNode* node = new ParseNode("<record-type>");  
  
    node→addChild(match(TokenType::RECORDSY));  
    node→addChild(parseFieldList());  
    node→addChild(match(TokenType::ENDSY));  
  
    return node;  
}  
  
ParseNode* Parser::parseFieldList() {  
    ParseNode* node = new ParseNode("<field-list>");  
  
    node→addChild(parseFieldPart());  
}
```

```

    while (peekToken().type == TokenType::SEMICOLON
&&
        peekToken(1).type != TokenType::ENDSY) {
        node→addChild(match(TokenType::SEMICOLON));
        node→addChild(parseFieldPart());
    }

    if (peekToken().type == TokenType::SEMICOLON) {
        node→addChild(match(TokenType::SEMICOLON));
    }

    return node;
}

```

parseRecordType() mengonsumsi token RECORDSY, memanggil parseFieldList(), kemudian mengonsumsi ENDSY. parseFieldList() menangani kemungkinan adanya trailing semicolon sebelum end dengan memeriksa lookahead satu token ke depan (peekToken(1) != ENDSY) sebelum melanjutkan loop.

2.6. Parser_ControlFlow.cpp

File ini mengimplementasikan parsing untuk deklarasi subprogram serta seluruh struktur *control flow* dalam bahasa Arion. Let me let me

parseProcedureDeclaration() dan parseFunctionDeclaration()

```

ParseNode* Parser::parseProcedureDeclaration(){
    ParseNode* node = new
    ParseNode("<procedure-declaration>");

    node→addChild(match(TokenType::PROCEDURESY));
    node→addChild(match(TokenType::IDENT));

    if(peekToken().type == TokenType::LPARENT){
        node→addChild(parseFormalParameterList());
    }

    node→addChild(match(TokenType::SEMICOLON));
}

```

```

        node→addChild(parseBlock());
        node→addChild(match(TokenType:: SEMICOLON));

        return node;
    }

    ParseNode* Parser::parseFunctionDeclaration() {
        ParseNode* node = new
        ParseNode("<function-declaration>");

        node→addChild(match(TokenType:: FUNCTIONSY));
        node→addChild(match(TokenType:: IDENT));

        if (peekToken().type == TokenType:: LPARENT) {
            node→addChild(parseFormalParameterList());
        }

        node→addChild(match(TokenType:: COLON));
        node→addChild(match(TokenType:: IDENT));

        node→addChild(match(TokenType:: SEMICOLON));
        node→addChild(parseBlock());
        node→addChild(match(TokenType:: SEMICOLON));

        return node;
    }

```

Keduanya memetakan production rule procedure-declaration dan function-declaration. Perbedaan utama adalah parseFunctionDeclaration() mengonsumsi tambahan COLON dan IDENT setelah daftar parameter untuk mendeklarasikan tipe kembalian fungsi. Keduanya memanggil parseBlock() untuk memproses tubuh subprogram, dan masing-masing membutuhkan dua SEMICOLON: satu setelah header dan satu setelah block.

parseFormalParameterList() dan parseParameterGroup()

```

    ParseNode* Parser::parseFormalParameterList() {
        ParseNode* node = new
        ParseNode("<formal-parameter-list>");

```

```

        node→addChild(match(TokenType::LPARENT));
        node→addChild(parseParameterGroup());

        while (peekToken().type == TokenType::SEMICOLON)
        {
            node→addChild(match(TokenType::SEMICOLON));
            node→addChild(parseParameterGroup());
        }

        node→addChild(match(TokenType::RPARENT));

        return node;
    }

    ParseNode* Parser::parseParameterGroup() {
        ParseNode* node = new
        ParseNode("<parameter-group>");

        node→addChild(parseIdentifierList());
        node→addChild(match(TokenType::COLON));

        if (peekToken().type == TokenType::ARRAYSY) {
            node→addChild(parseArrayType());
        } else {
            node→addChild(match(TokenType::IDENT));
        }

        return node;
    }

```

parseFormalParameterList() memproses daftar parameter formal yang dibungkus tanda kurung. parseParameterGroup() mendukung parameter bertipe named identifier atau bertipe array-type, dengan dispatch berdasarkan apakah token saat ini adalah ARRAYSY.

parseIfStatement()

```

    ParseNode* Parser::parseIfStatement() {

```

```

    ParseNode* node = new
    ParseNode("<if-statement>");

    node→addChild(match(TokenType::IFSY));
    node→addChild(parseExpression());
    node→addChild(match(TokenType::THENSY));
    node→addChild(parseStatement());

    if (peekToken().type == TokenType::ELSESY) {
        node→addChild(match(TokenType::ELSESY));
        node→addChild(parseStatement());
    }

    return node;
}

```

Memetakan production rule if-statement -> ifsy + expression + then + statement + (elsesy + statement)?. Keberadaan klausa else ditentukan dengan memeriksa apakah token saat ini ELSESY setelah statement then selesai diparsing.

parseCaseStatement() dan parseCaseBlock()

```

ParseNode* Parser::parseCaseStatement() {
    ParseNode* node = new
    ParseNode("<case-statement>");

    node→addChild(match(TokenType::CASESY));
    node→addChild(parseExpression());
    node→addChild(match(TokenType::OFSY));
    node→addChild(parseCaseBlock());
    node→addChild(match(TokenType::ENDSY));

    return node;
}

ParseNode* Parser::parseCaseBlock() {
    ParseNode* node = new ParseNode("<case-block>");

    node→addChild(parseConstant());
}

```

```

while (peekToken().type == TokenType::COMMA) {
    node→addChild(match(TokenType::COMMA));
    node→addChild(parseConstant());
}

node→addChild(match(TokenType::COLON));
node→addChild(parseStatement());

while (peekToken().type == TokenType::SEMICOLON)
{
    TokenType nextType = peekToken(1).type;
    bool canStartCaseBlock = (
        nextType == TokenType::INTCON ||
        nextType == TokenType::REALCON ||
        nextType == TokenType::CHARCON ||
        nextType == TokenType::STRING ||
        nextType == TokenType::IDENT ||
        nextType == TokenType::PLUS ||
        nextType == TokenType::MINUS
    );

    if (!canStartCaseBlock) break;

    node→addChild(match(TokenType::SEMICOLON));
    node→addChild(parseCaseBlock());
}

return node;
}

```

parseCaseStatement() memproses keseluruhan struktur case dari casesy hingga endsy. parseCaseBlock() memproses satu atau lebih entri case yang dipisahkan semicolon. Untuk menentukan apakah semicolon berikutnya memulai case baru atau merupakan terminator, dilakukan pengecekan token setelah semicolon (peekToken(1)) untuk melihat apakah dapat mengawali sebuah constant.

parseWhileStatement(), parseRepeatStatement(), parseForStatement()

```

ParseNode* Parser::parseWhileStatement() {
    ParseNode* node = new
ParseNode("<while-statement>");

    node→addChild(match(TokenType::WHILESY));
    node→addChild(parseExpression());
    node→addChild(match(TokenType::DOSY));
    node→addChild(parseStatement());

    return node;
}

ParseNode* Parser::parseRepeatStatement() {
    ParseNode* node = new
ParseNode("<repeat-statement>");

    node→addChild(match(TokenType::REPEATSY));
    node→addChild(parseStatementList());
    node→addChild(match(TokenType::UNTILSY));
    node→addChild(parseExpression());

    return node;
}

ParseNode* Parser::parseForStatement() {
    ParseNode* node = new
ParseNode("<for-statement>");

    node→addChild(match(TokenType::FORSY));
    node→addChild(match(TokenType::IDENT));
    node→addChild(match(TokenType::BECOMES));
    node→addChild(parseExpression());

    // (tosy | downtosy) – arah iterasi, wajib salah
    satu
    if (peekToken().type == TokenType::TOSY) {
        node→addChild(match(TokenType::TOSY));
    } else if (peekToken().type ==

```

```

TokenType::DOWNTOSY) {
    node→addChild(match(TokenType::DOWNTOSY));
} else {
    reportError("Expected 'to' or 'downto' in
for-statement, got: " + peekToken().toString());
    // Graceful fallback: buat node error tapi
jangan crash
    node→addChild(new ParseNode("ERROR(expected
to/downto)"));
}

    node→addChild(parseExpression());
    node→addChild(match(TokenType::DOSY));
    node→addChild(parseStatement());

    return node;
}

```

Ketiganya memetakan production rule masing-masing secara langsung. `parseForStatement()` menambahkan error handling khusus untuk memastikan terdapat token TOSY atau DOWNTOSY; jika tidak ditemukan, sebuah node ERROR dengan pesan deskriptif dibuat sebagai fallback tanpa menyebabkan crash.

2.7. Parser_Expression.cpp

File ini menangani hierarki operator dan ekspresi, serta node-node terkait variabel dan pemanggilan fungsi. Hierarki presedensi operator diimplementasikan secara implisit melalui urutan pemanggilan fungsi: `parseFactor()` → `parseTerm()` → `parseSimpleExpression()` → `parseExpression()`. Urutan ini memastikan bahwa operator dengan presedensi lebih tinggi (*, div, mod, and) dievaluasi sebelum operator presedensi lebih rendah (+, -, or), dan keduanya sebelum operator relasional (==, <>, <, <=, >, >=).

`parseAssignmentStatement()`

```

ParseNode* Parser::parseAssignmentStatement(){
    ParseNode* node = new
ParseNode("<assignment-statement>");

```

```

    // node→addChild(match(TokenType::IDENT));
    node→addChild(parseVariable());
    node→addChild(match(TokenType::BECOMES)); // :=
    node→addChild(parseExpression());
    return node;
}

```

Fungsi ini mengimplementasikan aturan grammar `<assignment-statement> → <variable> becomes <expression>`. Berbeda dari versi awal yang langsung `match(IDENT)`, versi final memanggil `parseVariable()` terlebih dahulu sebagai sisi kiri assignment. Hal ini memungkinkan parser menangani assignment ke bentuk variabel yang lebih kompleks seperti elemen array (`arr[i] := 5`) dan field record (`rec.nama := 'x'`), bukan hanya identifier tunggal. Setelah sisi kiri terparsing, token `:=` dikonsumsi, lalu `parseExpression()` dipanggil untuk memproses sisi kanan.

`parseProcedureFunctionCall()`

```

ParseNode* Parser::parseProcedureFunctionCall(){
    ParseNode* node = new
    ParseNode("<procedure/function-call>");

    // harus diawali dengan identifier (nama fungsi
    atau prosedur)
    node→addChild(match(TokenType::IDENT));

    if (peekToken().type == TokenType::LPARENT) {
        node→addChild(match(TokenType::LPARENT));
        node→addChild(parseParameterList()); //
    Parse isi parameter
        node→addChild(match(TokenType::RPARENT));
    }

    return node;
}

```

Fungsi ini mengimplementasikan aturan `<procedure/function-call> → ident (lparent parameter-list? rparent)?`. Pemanggilan prosedur atau fungsi diawali dengan nama (identifier), diikuti secara opsional oleh daftar argumen dalam

kurung. Opsionalitas ini diimplementasikan dengan if (`peekToken().type == LPARENT`). Jika tidak ada kurung buka, fungsi dianggap dipanggil tanpa parameter (seperti `readln` atau `nilaiAwal`). Perlu diperhatikan bahwa fungsi ini tidak menangani kasus parameter kosong seperti `prosedur()`. Jika `LPARENT` ditemukan, `parseParameterList()` langsung dipanggil tanpa cek apakah argumen ada atau tidak. Ini berarti `prosedur()` dengan parameter kosong akan menyebabkan error karena `parseParameterList()` selalu mengharapkan minimal satu ekspresi.

`parseParameterList()`

```
ParseNode* Parser::parseParameterList(){
    ParseNode* node = new
    ParseNode("<parameter-list>");

    // Parsing ekspresi / argumen pertama
    node→addChild(parseExpression());

    // Selama masih ada argumen lain yang dipisahkan
    koma
    while (peekToken().type == TokenType::COMMA) {
        node→addChild(match(TokenType::COMMA));
        node→addChild(parseExpression());
    }

    return node;
}
```

Fungsi ini mengimplementasikan aturan `<parameter-list> → expression (comma expression)*`. Ini adalah daftar argumen aktual saat pemanggilan prosedur atau fungsi, berbeda dari `<formal-parameter-list>` yang adalah daftar parameter dalam deklarasi. Pola implementasinya klasik untuk aturan EBNF dengan simbol `*`: satu elemen wajib diparsing terlebih dahulu (ekspresi pertama), kemudian selama ada `COMMA`, lanjutkan mengonsumsi koma dan ekspresi berikutnya. Setiap argumen dapat berupa ekspresi arbitrari, yaitu literal, variabel, operasi aritmatika, bahkan pemanggilan fungsi lain.

`parseExpression()`

```

ParseNode* Parser::parseExpression(){
    ParseNode* node = new ParseNode("<expression>");

    // simple term di awal
    node→addChild(parseSimpleExpression());

    // jika ada operator relasi, misal =,<,>,≤,≥
    TokenType cur = peekToken().type;
    if (cur == TokenType::EQL || cur ==
        TokenType::NEQ || cur == TokenType::LSS ||
        cur == TokenType::LEQ || cur ==
        TokenType::GTR || cur == TokenType::GEQ) {

        node→addChild(parseRelationalOperator());
        node→addChild(parseSimpleExpression());
    }

    return node;
}

```

Fungsi ini mengimplementasikan aturan $\langle \text{expression} \rangle \rightarrow \text{simple-expression} (\text{relational-operator simple-expression})?$. Ini adalah level tertinggi dalam hierarki ekspresi. Operator relasional ($=$, $<$, $>$, $\leq$, $\geq$) memiliki presedensi terendah dari semua operator. Mereka mengikat paling longgar, artinya seluruh sisi kiri dan kanan harus selesai dievaluasi sebelum perbandingan dilakukan. Opsionalitas operator relasional diimplementasikan dengan blok if yang mengecek enam kemungkinan tipe token sekaligus. Jika tidak ada operator relasional, ekspresi cukup berupa satu $\langle \text{simple-expression} \rangle$ saja seperti pada $a := 5 + 3$.

parseSimpleExpression()

```

ParseNode* Parser::parseSimpleExpression(){
    ParseNode* node = new
    ParseNode("<simple-expression>");

    // Cek opsional unary sign di awal ekspresi
    (misal: -5 atau +10)

```

```

        TokenType cur = peekToken().type;
        if (cur == TokenType::PLUS || cur ==
TokenType::MINUS) {
            node→addChild(match(cur));
        }

        // term
        node→addChild(parseTerm());

        // Selama token berikutnya adalah additive
        operator (+, -, OR)
        cur = peekToken().type;
        while (cur == TokenType::PLUS || cur ==
TokenType::MINUS || cur == TokenType::ORSY) {
            node→addChild(parseAdditiveOperator());
            node→addChild(parseTerm());
            cur = peekToken().type;
        }

        return node;
    }
}

```

Fungsi ini mengimplementasikan aturan $\langle \text{simple-expression} \rangle \rightarrow (\text{plus} \mid \text{minus})? \text{term} (\text{additive-operator term})^*$. Di sinilah operator penjumlahan, pengurangan, dan or ditangani. Tanda + atau - opsional di awal menangani kasus seperti ekspresi -a atau +b di mana a dan b adalah identifier, perlu dibedakan dari unary minus pada literal angka yang sudah ditangani Lexer menjadi INTCON(-67). Loop while menangani rantai operator seperti $a + b - c + d$ yang semuanya berada pada level presedensi yang sama dan diasosiasikan dari kiri ke kanan. Nilai cur diperbarui di akhir setiap iterasi loop untuk mengecek apakah masih ada operator aditif berikutnya.

isMultiplicativeOperator()

```

static bool isMultiplicativeOperator(TokenType type)
{
    return type == TokenType::TIMES ||

```

```

        type = TokenType::RDIV    ||
        type = TokenType::IDIV    ||
        type = TokenType::IMOD    ||
        type = TokenType::ANDSY;
    }

```

Ini bukan method kelas Parser melainkan fungsi helper statis yang hanya terlihat dalam file `Parser_Expression.cpp` (lingkup file). Fungsi ini mengenkapsulasi logika pengecekan apakah sebuah `TokenType` merupakan salah satu dari lima operator perkalian yang valid: `*` (perkalian), `/` (pembagian riil), `div` (pembagian integer), `mod` (modulo), dan `and` (konjungsi logika). Digunakan oleh `parseTerm()` sebagai kondisi loop `while` untuk menjaga kode tetap bersih dan mudah dibaca. Kata kunci `static` memastikan fungsi ini tidak terekspos ke unit kompilasi lain.

`parseTerm()`

```

ParseNode* Parser::parseTerm(){
    ParseNode* node = new ParseNode("<term>");

    // factor
    node->addChild(parseFactor());

    // (multiplicative-operator + factor)*
    while
(isMultiplicativeOperator(peekToken().type)) {
node->addChild(parseMultiplicativeOperator());
    node->addChild(parseFactor());
    }

    return node;
}

```

Fungsi ini mengimplementasikan aturan `<term> → factor (multiplicative-operator factor)*`. Ini menangani operator dengan presedensi lebih tinggi dari operator aditif: `*`, `/`, `div`, `mod`, dan `and`. Posisi `parseTerm()` di bawah `parseSimpleExpression()` dalam hierarki pemanggilan adalah yang menjamin presedensi operator bekerja dengan benar secara implisit. `2 + 3 * 4` akan

menghasilkan pohon di mana $3 * 4$ berada satu level lebih dalam (di `<term>`) dibanding `+` (di `<simple-expression>`), sehingga secara semantik perkalian dievaluasi terlebih dahulu. Loop while menggunakan helper `isMultiplicativeOperator()` untuk kondisinya.

`parseFactor()`

```
parseNode* Parser::parseFactor(){
    ParseNode* node = new ParseNode("<factor>");
    TokenType cur = peekToken().type;

    if (cur == TokenType::IDENT) {
        if (peekToken(1).type == TokenType::LPARENT)
        {
            node→addChild(parseProcedureFunctionCall());
        } else {
            node→addChild(parseVariable());
        }
    }
    // Literal konstanta
    else if (cur == TokenType::INTCON || cur ==
TokenType::REALCON ||
            cur == TokenType::STRING || cur ==
TokenType::CHARCON) {
        node→addChild(match(cur));
    }
    // Ekspresi di dalam kurung ( )
    else if (cur == TokenType::LPARENT) {
        node→addChild(match(TokenType::LPARENT));
        node→addChild(parseExpression());
        node→addChild(match(TokenType::RPARENT));
    }
    // Format pertidaksamaan boolean (NOT)
    else if (cur == TokenType::NOTSY) {
        node→addChild(match(TokenType::NOTSY));
        node→addChild(parseFactor()); // Rekursif ke
factor berikutnya yang dinegasikan
    }
```

```

    }
    else {
        // error handling
        reportError("Expected Factor (ident, literal,
'(' or NOT");
        node→addChild(new ParseNode("ERROR"));
        advToken();
    }

    return node;
}

```

Fungsi ini mengimplementasikan aturan <factor> yang merupakan unit terkecil dan tidak dapat dibagi lagi dalam hierarki ekspresi. Ada lima cabang utama yang dibedakan menggunakan lookahead:

- Cabang IDENT: Menggunakan lookahead dua token (peekToken(1)) untuk membedakan dua kemungkinan. Jika token setelah identifier adalah LPARENT, ini adalah pemanggilan fungsi (seperti sqrt(x)) sehingga dipanggil parseProcedureFunctionCall(). Jika bukan, ini adalah penggunaan variabel biasa (termasuk akses array/record) sehingga dipanggil parseVariable().
- Cabang literal: Menangani empat jenis konstanta literal langsung: integer (INTCON), real (REALCON), string (STRING), dan karakter (CHARCON). Cukup dikonsumsi dengan match(cur) dan menjadi leaf node.
- Cabang ekspresi dalam kurung: Menangani subekspresi yang diapit tanda kurung seperti (a + b) * c. LPARENT dan RPARENT dikonsumsi sebagai node struktural, sementara isi di dalamnya diproses dengan memanggil kembali parseExpression(). Ini adalah titik di mana rekursi mutual antara parseFactor() dan parseExpression() terjadi, memungkinkan ekspresi bersarang sedalam apapun.
- Cabang NOT: Menangani negasi logika seperti not (x > 0). Setelah NOT dikonsumsi, parseFactor() dipanggil secara rekursif untuk memproses operand-nya. Rekursi ini memungkinkan penulisan not not a yang valid secara sintaksis.
- Cabang error: Jika tidak ada cabang yang cocok, reportError() dipanggil dengan pesan deskriptif, sebuah node "ERROR" ditambahkan, dan

advToken() dipanggil untuk melewati token bermasalah guna mencegah infinite loop.

parseComponentVariable()

```
ParseNode* Parser::parseComponentVariable() {
    ParseNode* node = new
ParseNode("<component-variable>");

    if (peekToken().type == TokenType::LBRACK) {
        node→addChild(match(TokenType::LBRACK));
        node→addChild(parseIndexList());
        node→addChild(match(TokenType::RBRACK));
    }
    else if (peekToken().type == TokenType::PERIOD) {
        node→addChild(match(TokenType::PERIOD));
        node→addChild(match(TokenType::IDENT));
    }
    else {
        reportError("Expected '[' for array element
or '.' for record field");
        node→addChild(new ParseNode("ERROR"));
        advToken();
    }

    return node;
}
```

Fungsi ini mengimplementasikan aturan <component-variable> → (lbrack index-list rbrack) | (period ident). Ia menangani dua cara mengakses bagian dari variabel kompleks. Cabang pertama (LBRACK) menangani akses elemen array seperti arr[i] atau matrix[i, j], kurung siku dibuka, daftar indeks diparsing, lalu kurung siku ditutup. Cabang kedua (PERIOD) menangani akses field record seperti mahasiswa.nama. Titik dikonsumsi diikuti nama field sebagai identifier. Fungsi ini tidak dipanggil secara mandiri melainkan dipanggil berulang kali dari dalam loop di parseVariable().

parseIndexList()

```

ParseNode* Parser::parseIndexList(){
    ParseNode* node = new ParseNode("<index-list>");

    // ( intcon| charcon|ident)
    TokenType cur = peekToken().type;
    if (cur == TokenType::INTCON || cur ==
TokenType::CHARCON || cur == TokenType::IDENT) {
        node→addChild(match(cur));
    } else {
        reportError("Expected integer constant,
character constant, or identifier as array index");
        node→addChild(new ParseNode("ERROR"));
        advToken();
    }

    // +(comma + index-list )*
    while (peekToken().type == TokenType::COMMA) {
        node→addChild(match(TokenType::COMMA));

        cur = peekToken().type;
        if (cur == TokenType::INTCON || cur ==
TokenType::CHARCON || cur == TokenType::IDENT) {
            node→addChild(match(cur));
        } else {
            reportError("Expected integer constant,
character constant, or identifier as array index");
            node→addChild(new ParseNode("ERROR"));
            advToken();
        }
    }

    return node;
}

```

Fungsi ini mengimplementasikan aturan $\langle \text{index-list} \rangle \rightarrow (\text{intcon} \mid \text{charcon} \mid \text{ident}) (\text{comma} (\text{intcon} \mid \text{charcon} \mid \text{ident}))^*$. Ia menangani daftar indeks di dalam kurung siku array, yang bisa berupa satu atau lebih nilai yang dipisahkan koma untuk

array multidimensi seperti `matrix[2, 3]`. Tipe indeks yang diizinkan terbatas pada tiga jenis: `INTCON` (seperti `arr[1]`), `CHARCON` (seperti `charArr['a']`), dan `IDENT` (seperti `arr[i]`). Setiap posisi indeks memiliki validasi tersendiri, baik untuk indeks pertama maupun indeks-indeks berikutnya setelah koma, masing-masing dengan pesan error yang deskriptif jika token yang ditemukan tidak sesuai.

`parseVariable()`

```
ParseNode* Parser::parseVariable(){
    ParseNode* node = new ParseNode("<variable>");
    TokenType cur = peekToken().type;
    if(cur == TokenType::IDENT){
        node→addChild(match(cur));
        while(peekToken().type == TokenType::LBRACK
|| peekToken().type == TokenType::PERIOD){
            node→addChild(parseComponentVariable());
        }
    }
    else{
        reportError("Expected Identifier for
variable");
        node→addChild(new ParseNode("ERROR"));
        advToken();
    }
    return node;
}
```

Fungsi ini mengimplementasikan aturan `<variable> → ident (component-variable)*`. Ia adalah titik masuk untuk mengakses variabel dalam berbagai bentuk, dari yang paling sederhana hingga berantai. Setelah identifier awal dikonsumsi, loop while terus berjalan selama token berikutnya adalah `LBRACK` atau `PERIOD`, memanggil `parseComponentVariable()` setiap iterasi. Pola ini memungkinkan akses berantai seperti `records[i].mahasiswa.nama` yang akan menghasilkan satu node `<variable>` dengan tiga node `<component-variable>` sebagai anak-anaknya. Fungsi ini dipanggil dari `parseAssignmentStatement()`

untuk sisi kiri assignment dan dari `parseFactor()` untuk penggunaan variabel dalam ekspresi.

`parseRelationalOperator()`

```
ParseNode* Parser::parseRelationalOperator(){
    ParseNode* node = new
    ParseNode("<relational-operator>");
    node→addChild(match(peekToken().type)); // ini
    udah pasti eql, neq, gtr, geq, lss, leq
    return node;
}
```

Fungsi ini mengimplementasikan aturan `<relational-operator> → eql | neq | gtr | geq | lss | leq`. Ia mengonsumsi token operator relasional apapun yang saat ini berada di posisi current tanpa melakukan validasi tambahan. Validasi bahwa token memang merupakan operator relasional yang valid sudah dilakukan sebelumnya oleh `parseExpression()` melalui blok if yang mengecek enam kemungkinan tipe sebelum memanggil fungsi ini. Dengan kata lain, jika fungsi ini dipanggil, dijamin token saat ini adalah salah satu dari EQL, NEQ, GTR, GEQ, LSS, atau LEQ.

`parseAdditiveOperator()`

```
ParseNode* Parser::parseAdditiveOperator(){
    ParseNode* node = new
    ParseNode("<additive-operator>");
    node→addChild(match(peekToken().type)); // ini
    udah pasti plus, minus, orsy
    return node;
}
```

Fungsi ini mengimplementasikan aturan `<additive-operator> → plus | minus | orsy`. Polanya identik dengan `parseRelationalOperator()`, mengonsumsi token apapun yang saat ini ada tanpa validasi tambahan, karena validasi sudah dilakukan oleh kondisi while di `parseSimpleExpression()` yang mengecek PLUS, MINUS, atau ORSY sebelum memanggil fungsi ini. Operator or berada di level ini (bukan di `parseMultiplicativeOperator()`) karena dalam aljabar boolean, or

memiliki presedensi lebih rendah dari `and`, sama seperti `+` memiliki presedensi lebih rendah dari `*` dalam aritmatika.

`parseMultiplicativeOperator()`

```
ParseNode* Parser::parseMultiplicativeOperator(){
    ParseNode* node = new
    ParseNode("<multiplicative-operator>");
    node->addChild(match(peekToken().type)); // ini
    udh psti times | rdiv | idiv | imod | andsy
    return node;
}
```

Fungsi ini mengimplementasikan aturan `<multiplicative-operator> → times | rdiv | idiv | imod | andsy`. Polanya sama dengan dua fungsi operator di atas. Validasi dilakukan oleh kondisi `while` di `parseTerm()` melalui helper `isMultiplicativeOperator()`. Perlu diperhatikan bahwa `and` berada di level ini karena dalam Pascal dan bahasa-bahasa turunannya, `and` diperlakukan setara presedensinya dengan perkalian, sehingga `a or b and c` diinterpretasikan sebagai `a or (b and c)`. Ini konsisten dengan konvensi standar logika propositional.

2.8. Main.cpp

Entry point program mengintegrasikan seluruh komponen. Program menerima dua argumen command-line: nama file input dan nama file output, keduanya relatif terhadap direktori `test/milestone-2/`. Setelah konstruksi Lexer dan Parser, `parser.parse()` dipanggil untuk membangun parse tree. Jika `parser.isSuccess()` bernilai `true`, parse tree dicetak ke output file menggunakan `parseTreeRoot->printTree(out)`. Jika ada syntax error, pesan kegagalan dicetak dan program memberikan kode exit yang jelas. Memori parse tree dibersihkan dengan `delete parseTreeRoot` yang secara rekursif menghapus seluruh pohon melalui destruktur `ParseNode`.

BAB 3

PENGUJIAN

1. Test Case 1

input1.txt

```
program AdvancedTest;

const
  phi = 3.14;
  max_speed = 100;

type
  Colors = (Red, Green, Blue);
  Point = record
    x, y: integer;
  end;

var
  p1: Point;
  c: Colors;
  i: integer;

procedure MovePoint(dx, dy: integer);
begin
  p1.x := p1.x + dx;
  p1.y := p1.y + dy;
end;

begin
  p1.x := 0;
  p1.y := 0;
  c := Red;

  for i := 1 to 5 do
    MovePoint(i, i * 2);

  case i of
```

```

1, 2: writeln('Low');
3, 4: writeln('Medium');
5: writeln('High');
end;
end.

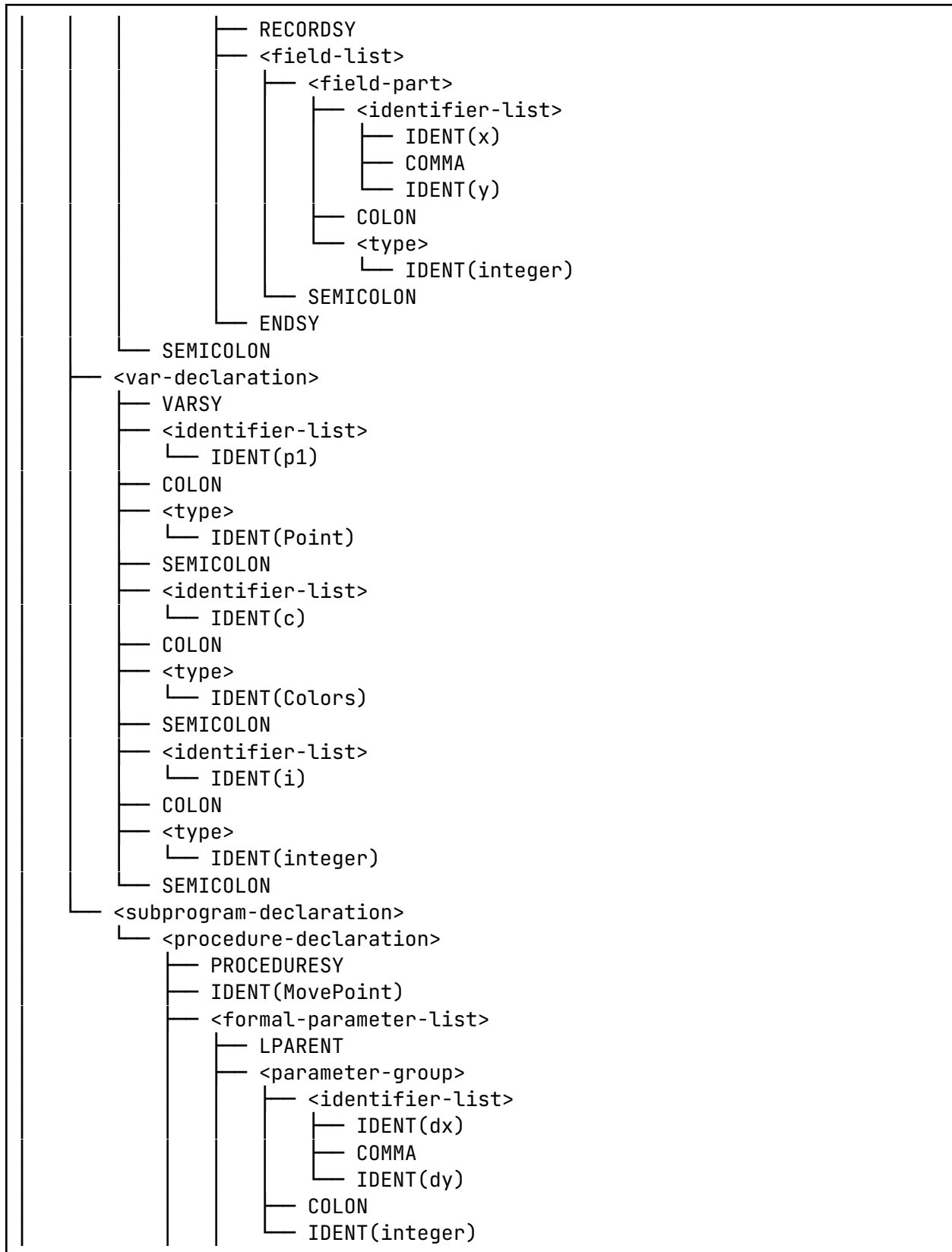
```

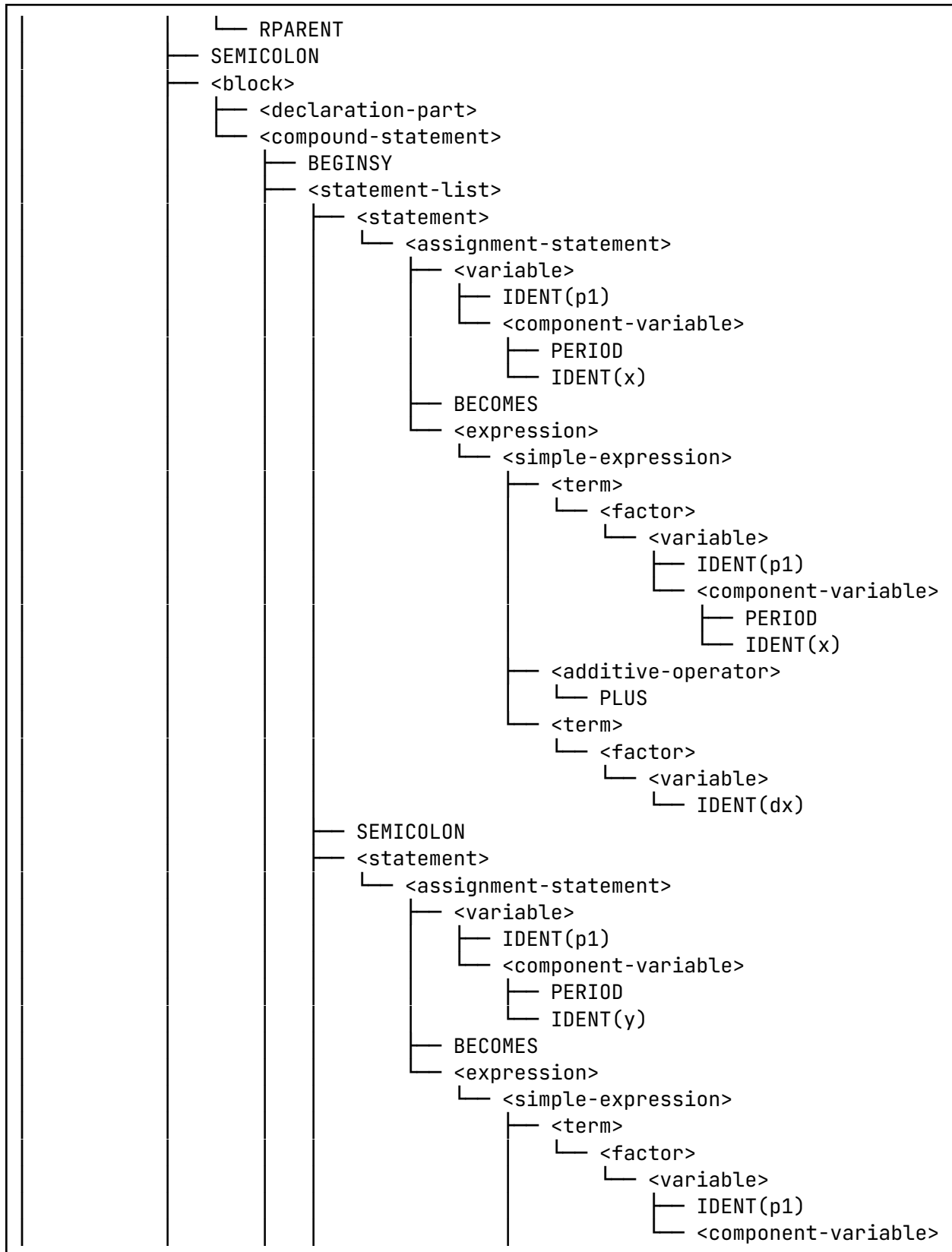
output1.txt

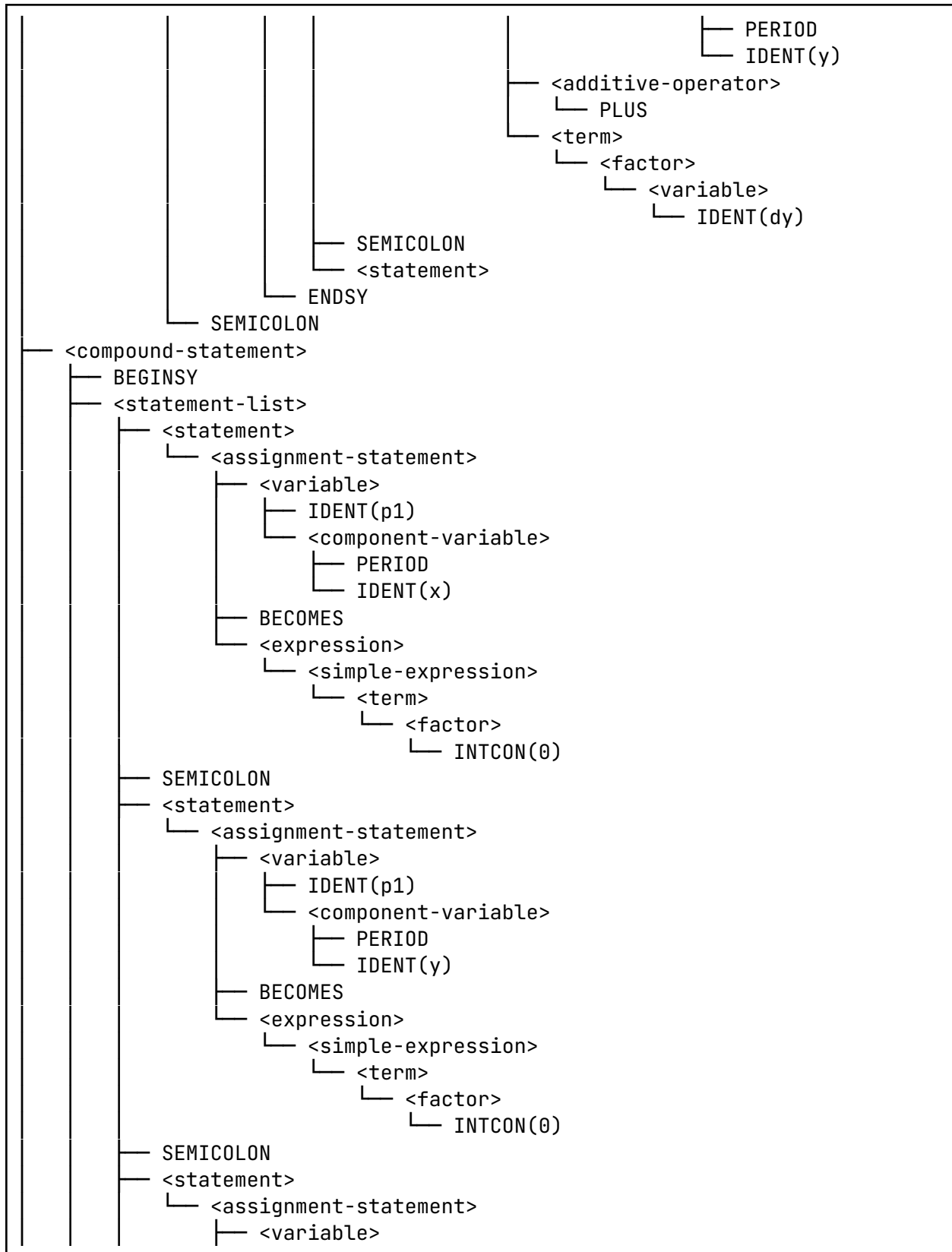
```

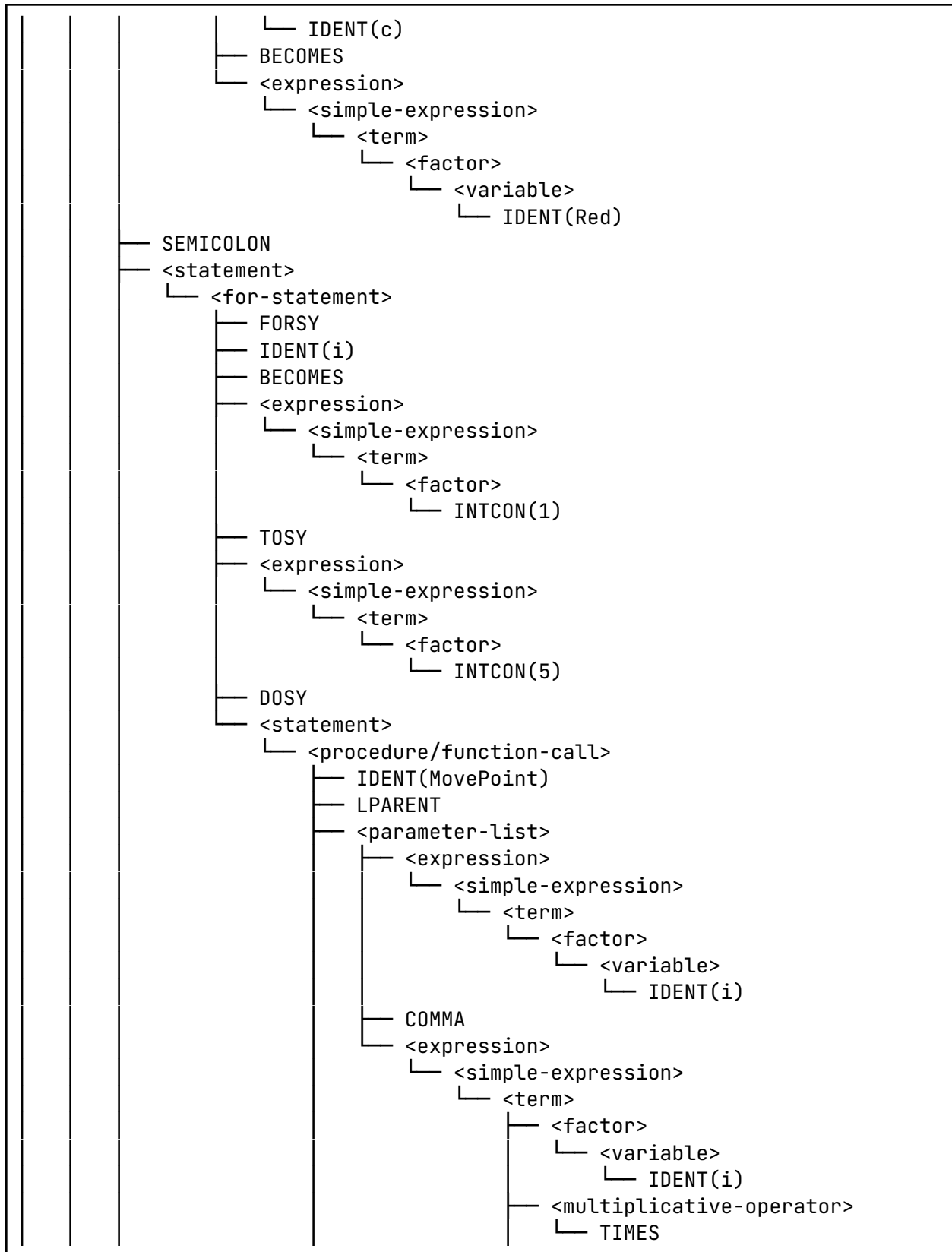
<program>
├── <program-header>
│   ├── PROGRAMSY
│   ├── IDENT(AdvancedTest)
│   └── SEMICOLON
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── CONSTSY
│   │   ├── IDENT(phi)
│   │   ├── EQL
│   │   ├── <constant>
│   │   │   └── REALCON(3.14)
│   │   ├── SEMICOLON
│   │   ├── IDENT(max_speed)
│   │   ├── EQL
│   │   ├── <constant>
│   │   │   └── INTCON(100)
│   │   └── SEMICOLON
│   ├── <type-declaration>
│   │   ├── TYPESY
│   │   ├── IDENT(Colors)
│   │   ├── EQL
│   │   ├── <type>
│   │   │   └── <enumerated>
│   │   │       ├── LPARENT
│   │   │       ├── IDENT(Red)
│   │   │       ├── COMMA
│   │   │       ├── IDENT(Green)
│   │   │       ├── COMMA
│   │   │       ├── IDENT(Blue)
│   │   │       └── RPARENT
│   │   ├── SEMICOLON
│   │   ├── IDENT(Point)
│   │   ├── EQL
│   │   ├── <type>
│   │   │   └── <record-type>

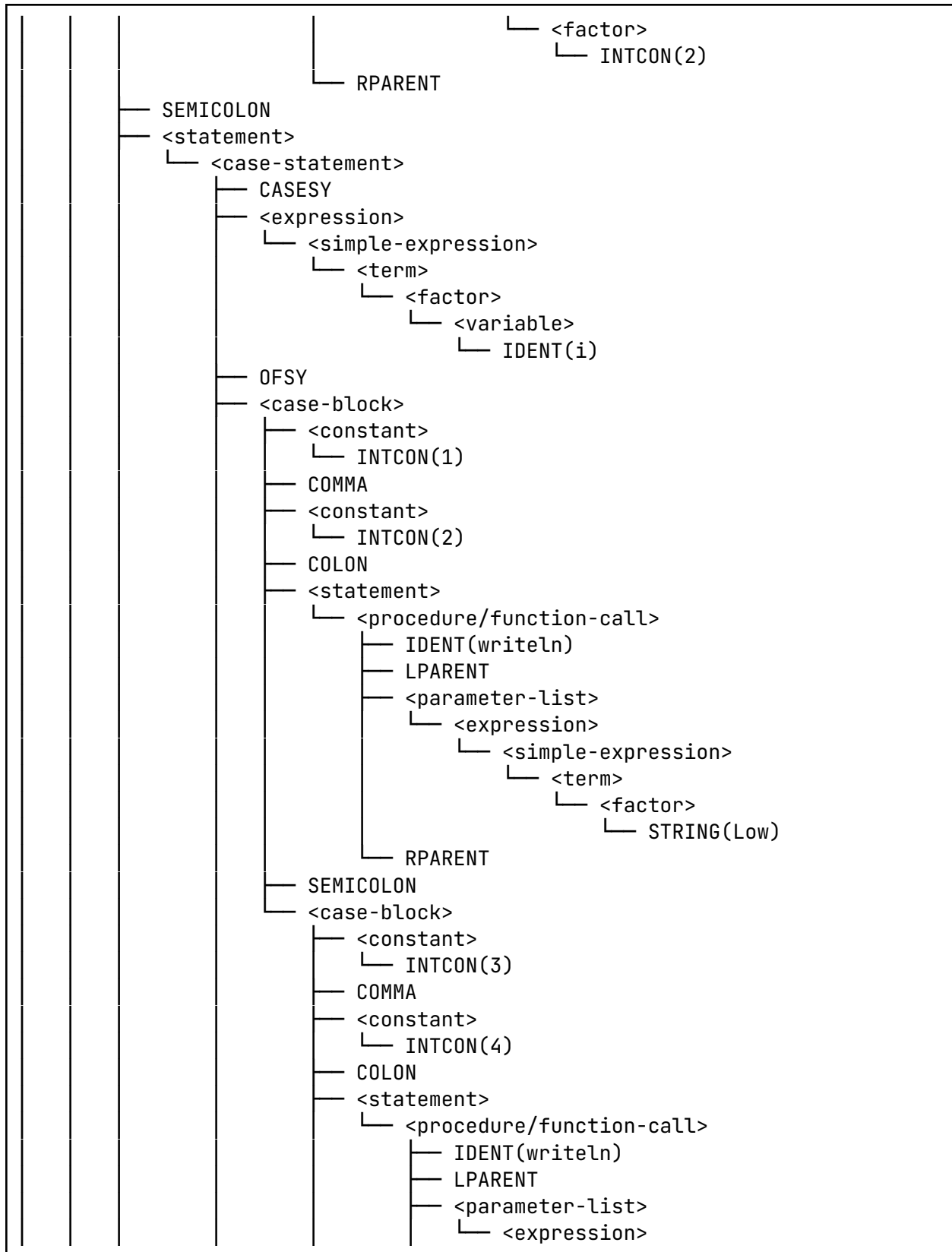
```

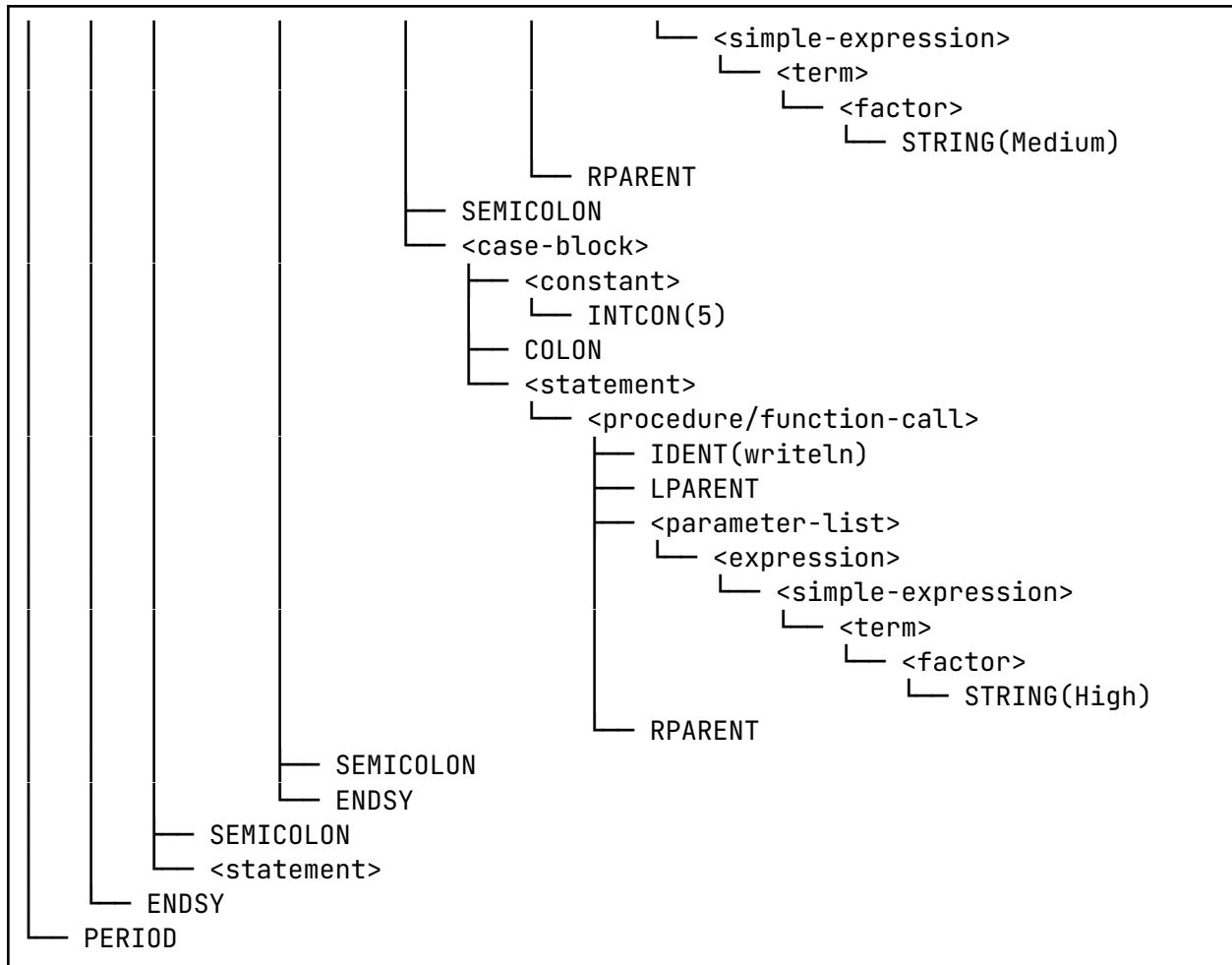












2. Test Case 2

input2.txt
<pre> program Hello; var a, b, c,d: integer; ArrayPonso: array [1..10] of integer; ArrayEdo: array [1..10] of char; begin a := 5; b := a + 10; writeln('Result = ', b); </pre>

```

c := 15;
d := c - a;
writeln('Result = ', d);
ArrayPonso[1] := 20;
ArrayEdo[1] := 'A';

if(ArrayPonso[1] > 10) then
  writeln('ArrayPonso[1] is greater than 10')
else
  writeln('ArrayPonso[1] is not greater than 10');
end.

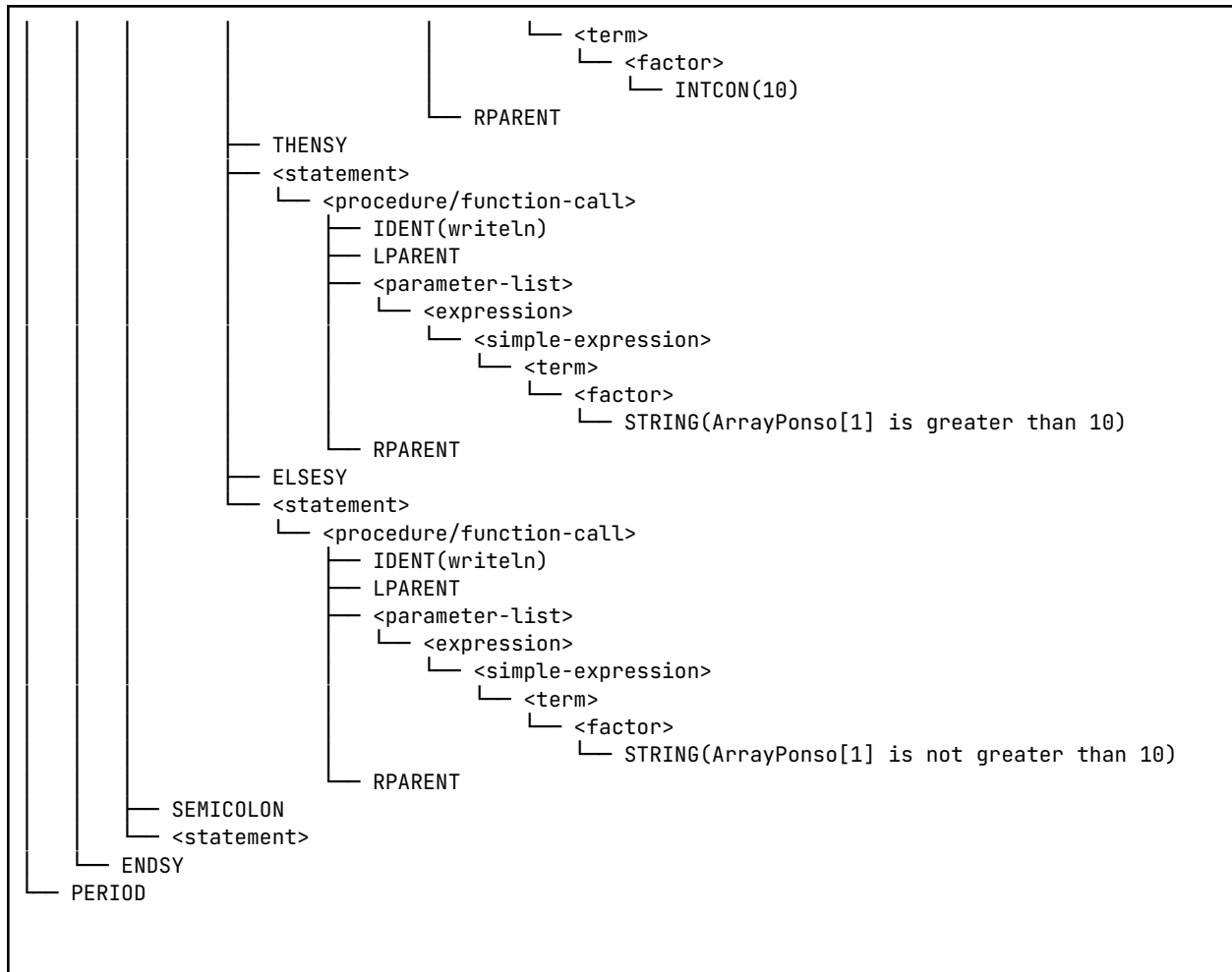
```

output2.txt

```

<program>
├── <program-header>
│   ├── PROGRAMSY
│   ├── IDENT(Hello)
│   └── SEMICOLON
├── <declaration-part>
│   └── <var-declaration>
│       ├── VARSY
│       ├── <identifier-list>
│       │   ├── IDENT(a)
│       │   ├── COMMA
│       │   ├── IDENT(b)
│       │   ├── COMMA
│       │   ├── IDENT(c)
│       │   ├── COMMA
│       │   └── IDENT(d)
│       ├── COLON
│       ├── <type>
│       │   └── IDENT(integer)
│       ├── SEMICOLON
│       ├── <identifier-list>
│       │   └── IDENT(ArrayPonso)
│       ├── COLON
│       ├── <type>
│       │   └── <array-type>
│       │       ├── ARRAYSY
│       │       ├── LBRACK
│       │       ├── <range>
│       │       │   ├── <constant>
│       │       │   │   └── INTCON(1)
│       │       │   ├── PERIOD
│       │       │   ├── PERIOD
│       │       │   └── <constant>
│       │       │       └── INTCON(10)
│       │       ├── RBRACK
│       │       └── OFSY

```

3. Test Case 3

input3.txt
<pre> program DataProcessing; var a, b, limit: integer; ArrayA, ArrayB: array [1..5] of integer; Kode: array [1..3] of char; begin a := 1; b := 5; limit := 5; </pre>

```

Kode[1] := 'A';
Kode[2] := 'B';
Kode[3] := 'C';

repeat
  ArrayA[a] := a * 2;
  ArrayB[a] := b * 2;

  if (ArrayA[a] > ArrayB[a]) then
    writeln('ArrayA menang dengan nilai ', ArrayA[a])
  else
    writeln('ArrayB menang dengan nilai ', ArrayB[a]);

  a := a + 1;
  b := b - 1;

  writeln('Kode eksekusi terakhir: ', Kode[3]);
until a > limit + 1
end.

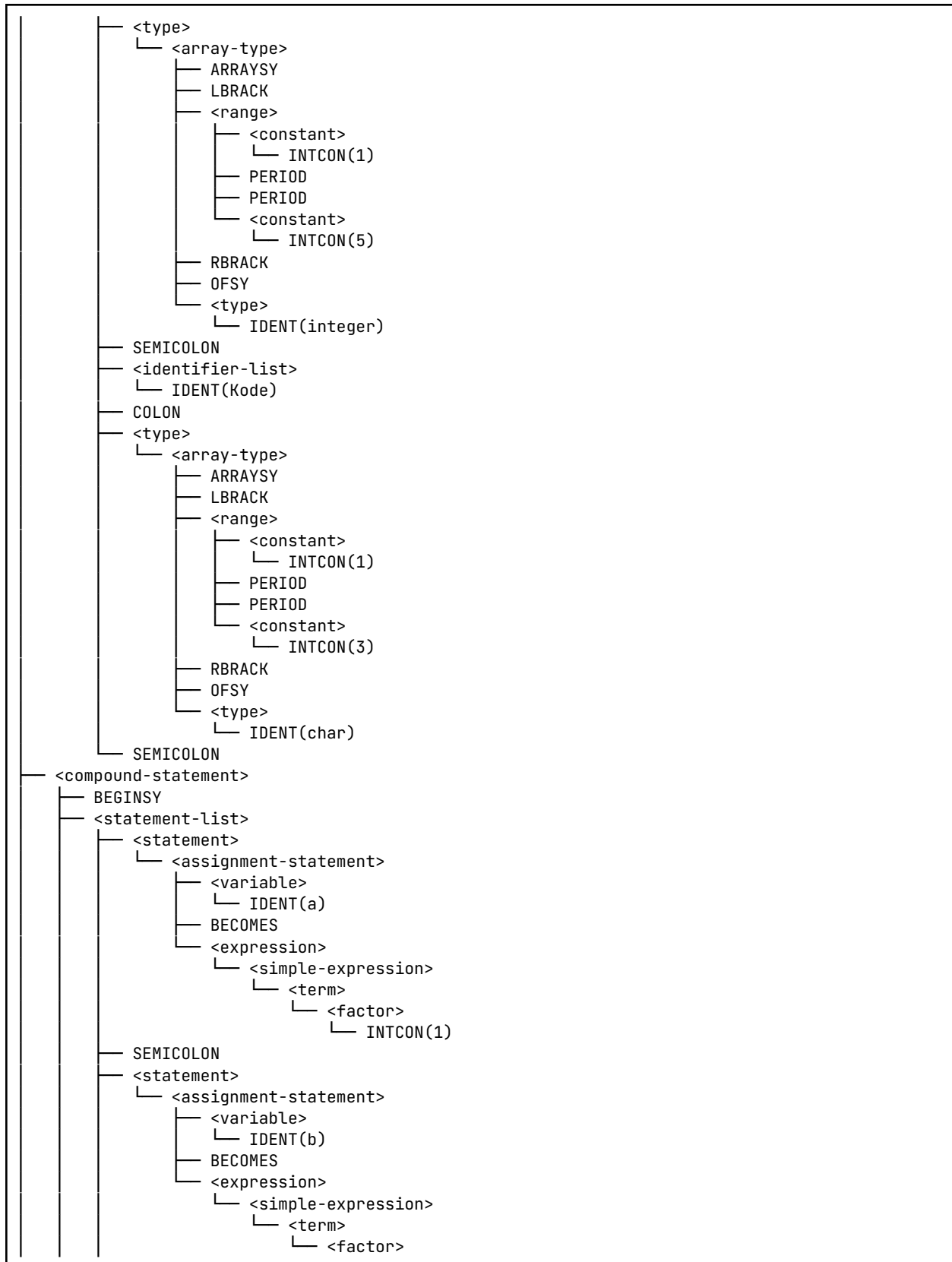
```

output3.txt

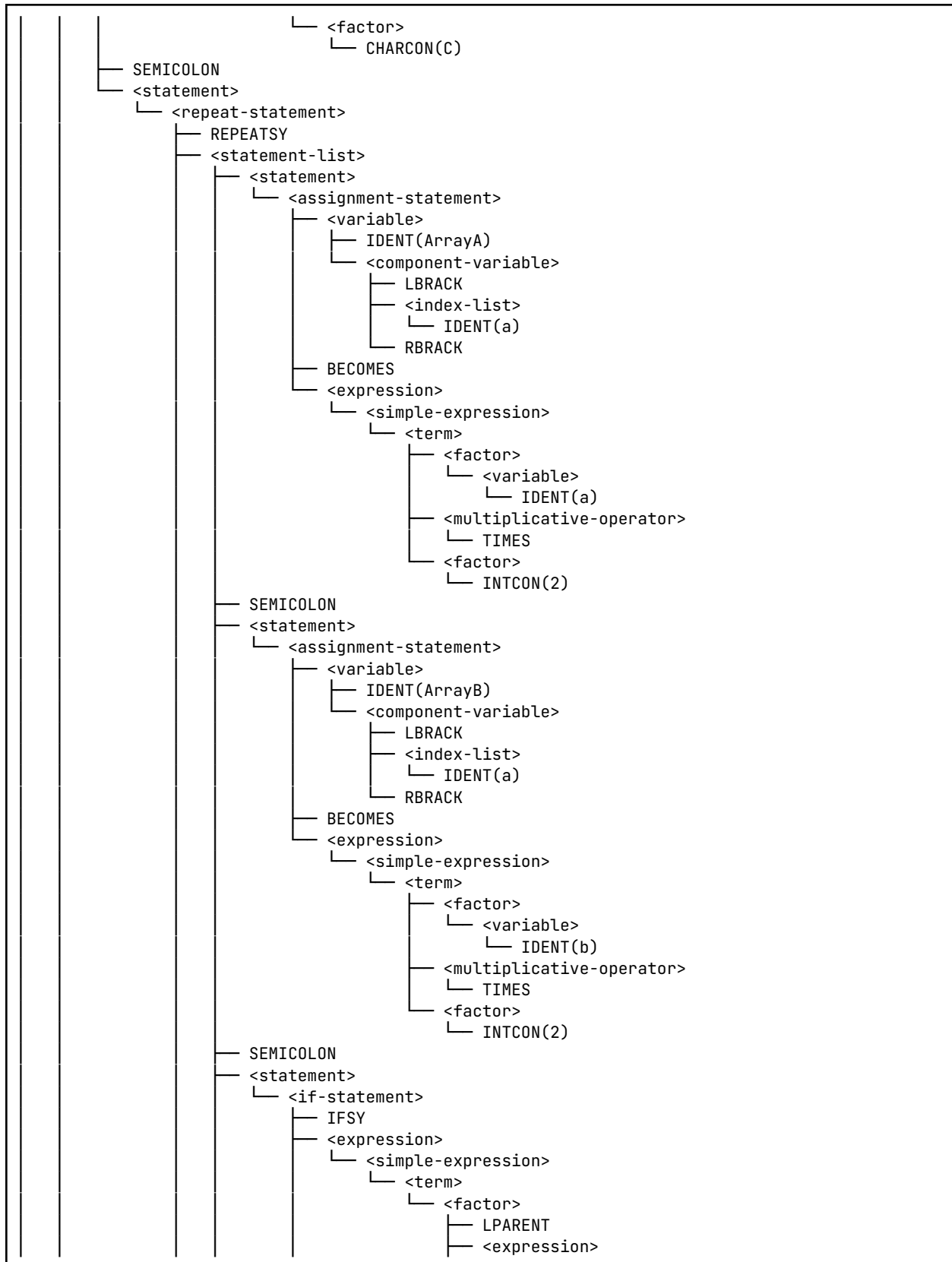
```

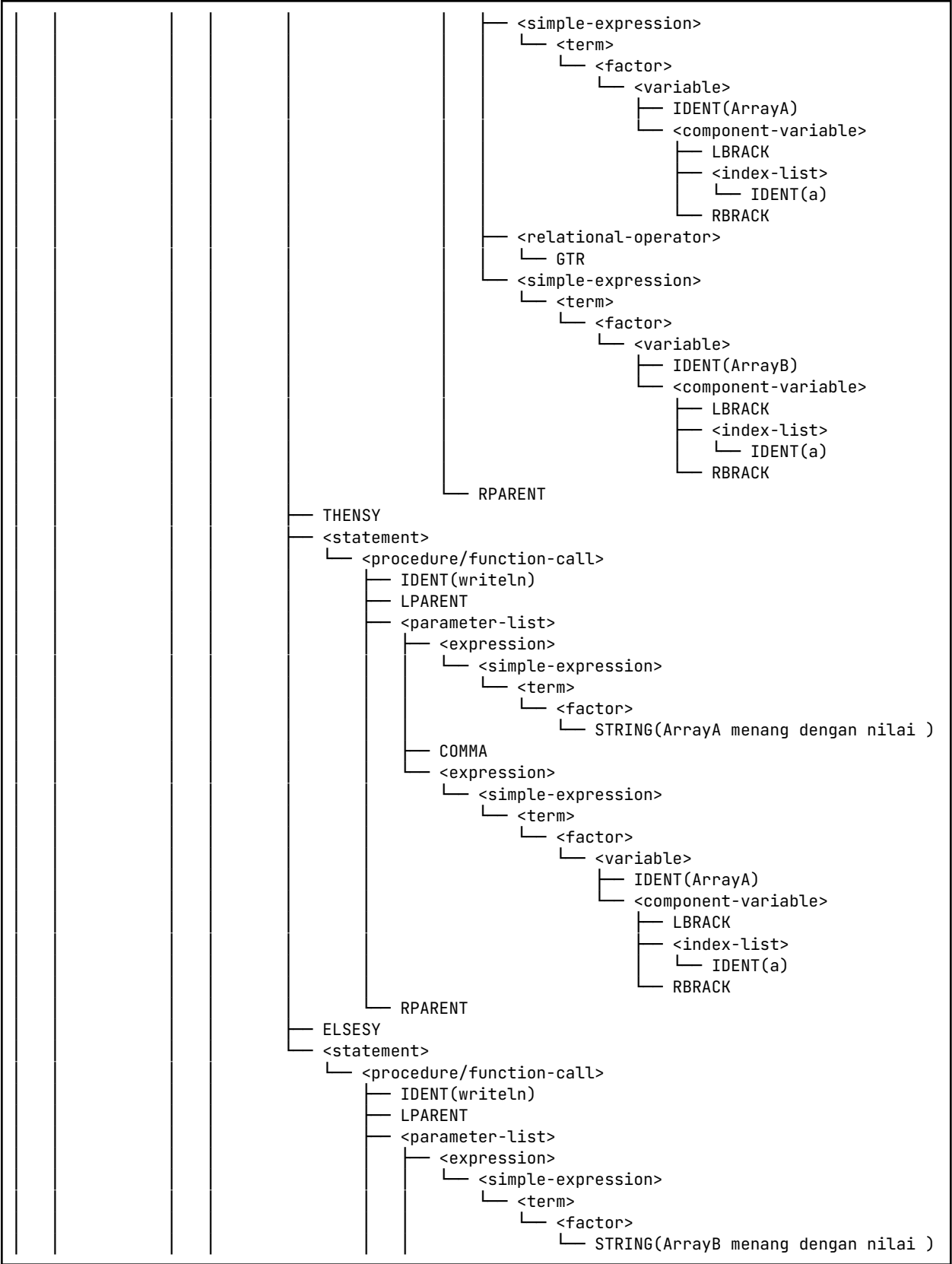
<program>
├── <program-header>
│   ├── PROGRAMSY
│   ├── IDENT(DataProcessing)
│   └── SEMICOLON
├── <declaration-part>
│   └── <var-declaration>
│       ├── VARSY
│       ├── <identifier-list>
│       │   ├── IDENT(a)
│       │   ├── COMMA
│       │   ├── IDENT(b)
│       │   ├── COMMA
│       │   └── IDENT(limit)
│       ├── COLON
│       ├── <type>
│       │   └── IDENT(integer)
│       ├── SEMICOLON
│       ├── <identifier-list>
│       │   ├── IDENT(ArrayA)
│       │   ├── COMMA
│       │   └── IDENT(ArrayB)
│       └── COLON

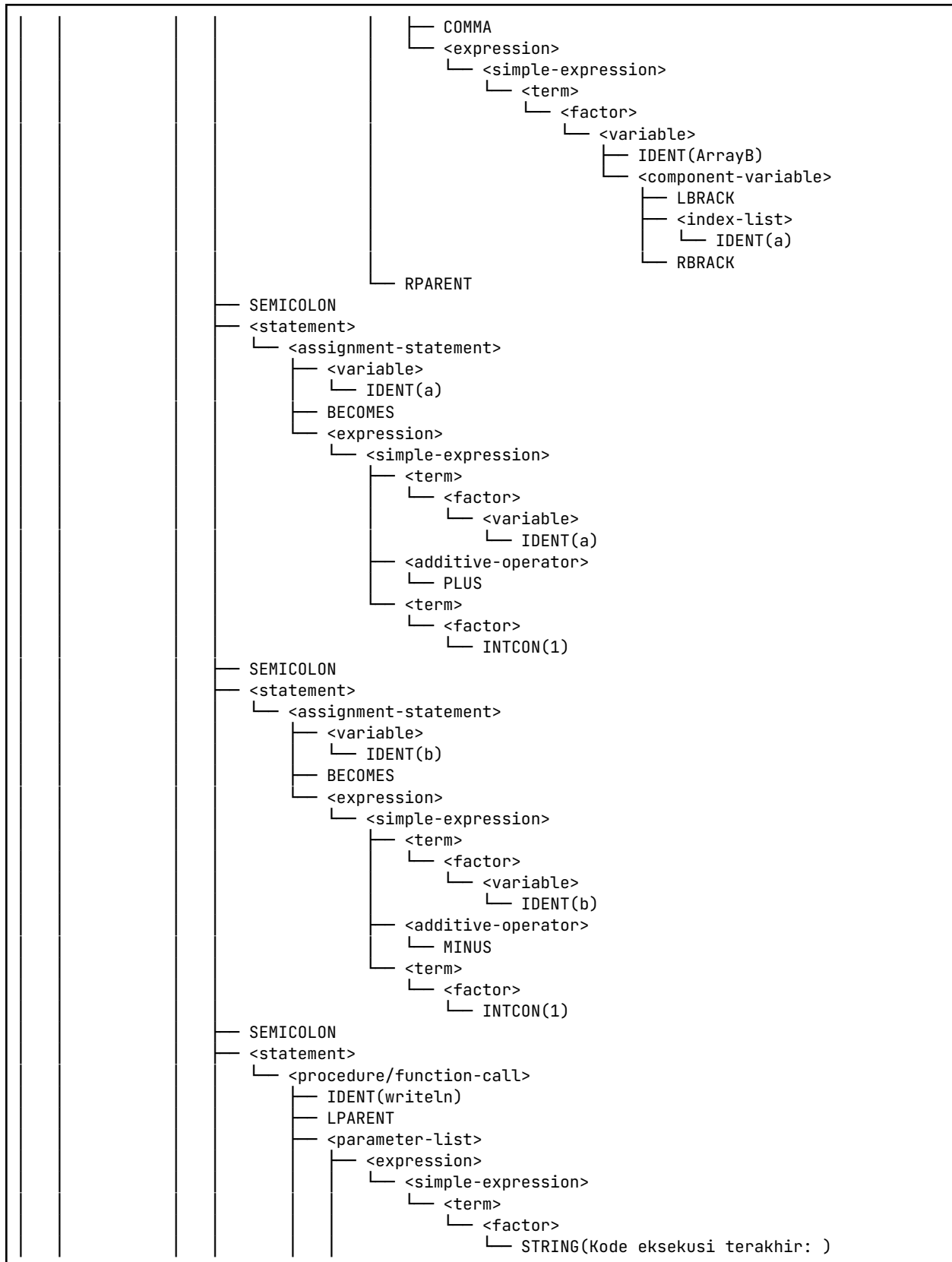
```

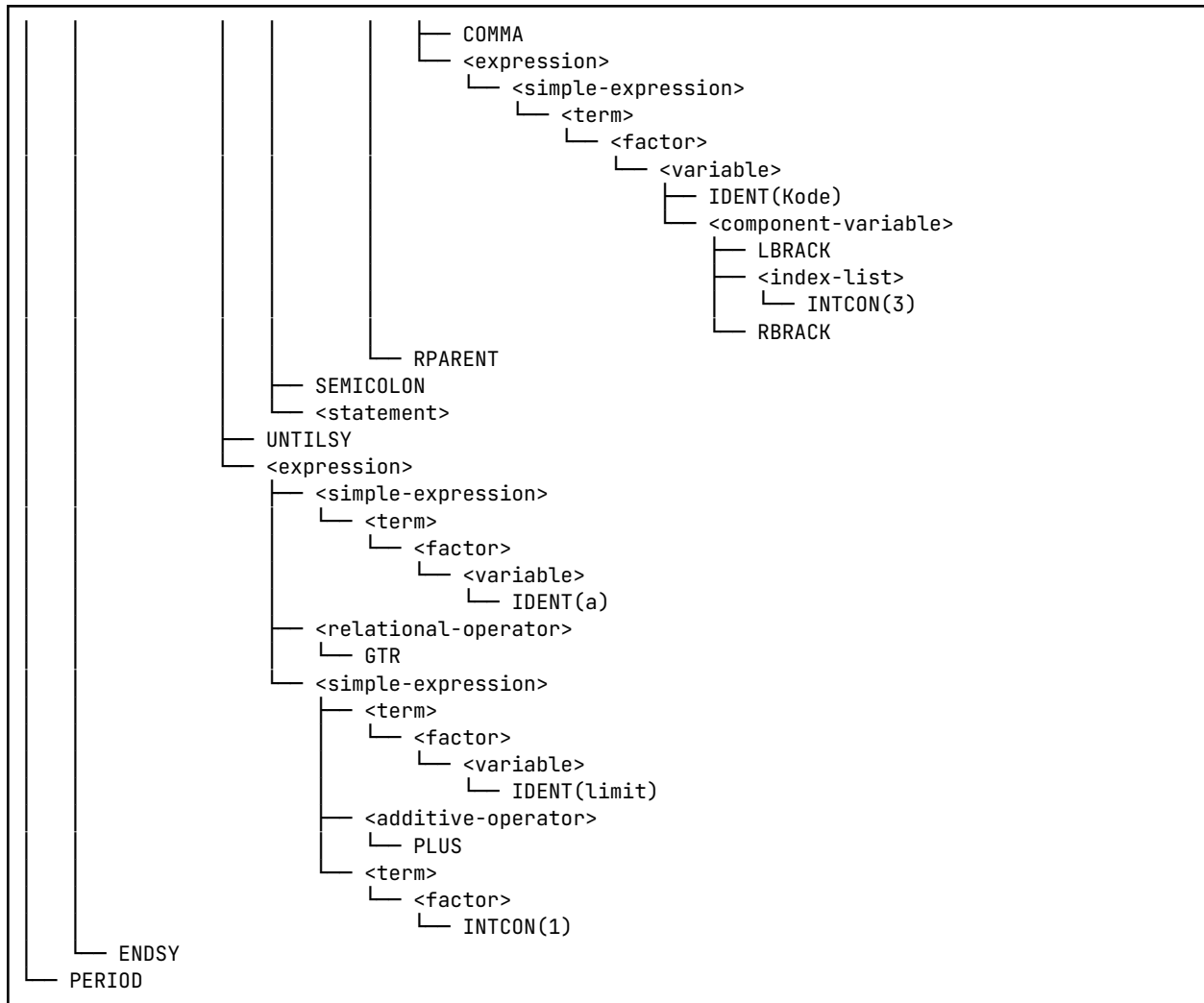












4. Test Case 4

input4.txt
<pre> program NestedLogic; var x, y, z, total: integer; status: char; begin x := 10; y := 5; z := 0; </pre>

```

total := 0;
status := 'N';

while x > 0 do
  z := x - y;
  if z > 2 then total := total + z
  else
    if z = 0 then total := total + 1
    else total := total - 1;
  x := x - 1;
  writeln('X: ', x, ' Z: ', z, ' Total: ', total, ' Status:
', status);
  x := x - 2;

  writeln('Eksekusi selesai dengan nilai akhir: ', total);
end.

```

output4.txt

```

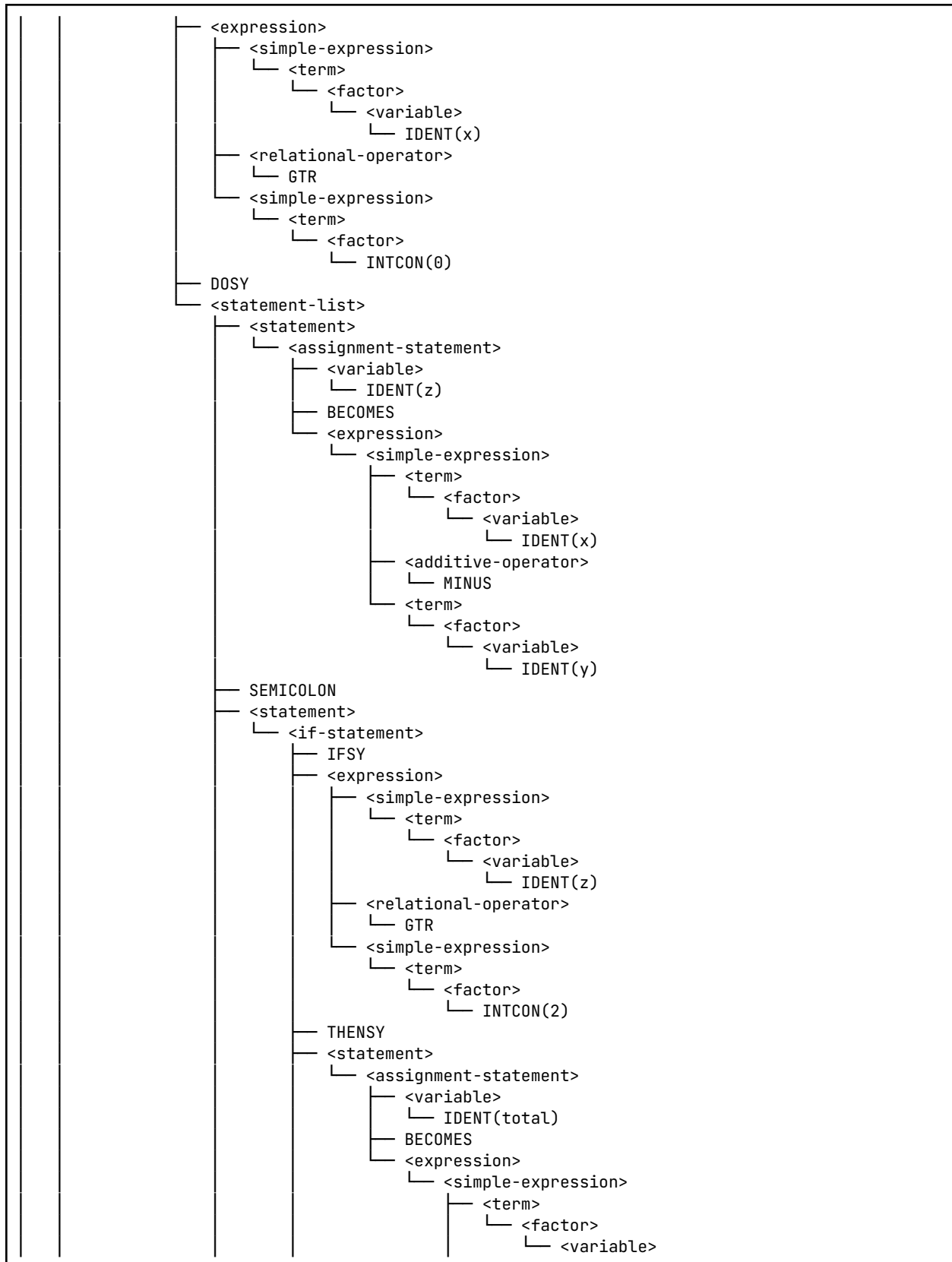
<program>
├── <program-header>
│   ├── PROGRAMSY
│   ├── IDENT(NestedLogic)
│   └── SEMICOLON
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── VARSY
│   │   ├── <identifier-list>
│   │   │   ├── IDENT(x)
│   │   │   ├── COMMA
│   │   │   ├── IDENT(y)
│   │   │   ├── COMMA
│   │   │   ├── IDENT(z)
│   │   │   ├── COMMA
│   │   │   └── IDENT(total)
│   │   ├── COLON
│   │   ├── <type>
│   │   │   └── IDENT(integer)
│   │   ├── SEMICOLON
│   │   ├── <identifier-list>
│   │   │   └── IDENT(status)
│   │   ├── COLON
│   │   ├── <type>
│   │   │   └── IDENT(char)
│   │   └── SEMICOLON
└── <compound-statement>
    ├── BEGINSY
    └── <statement-list>

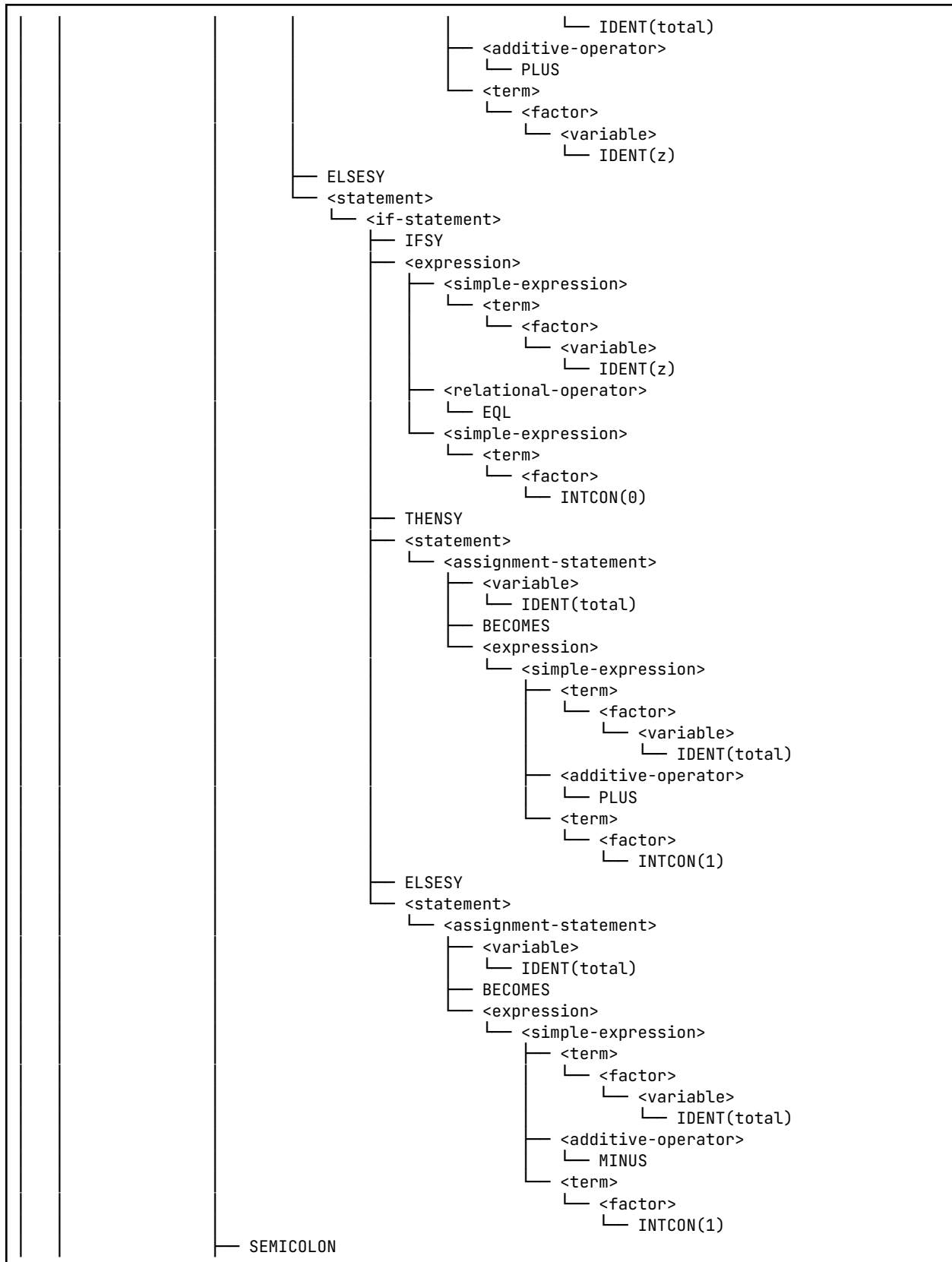
```

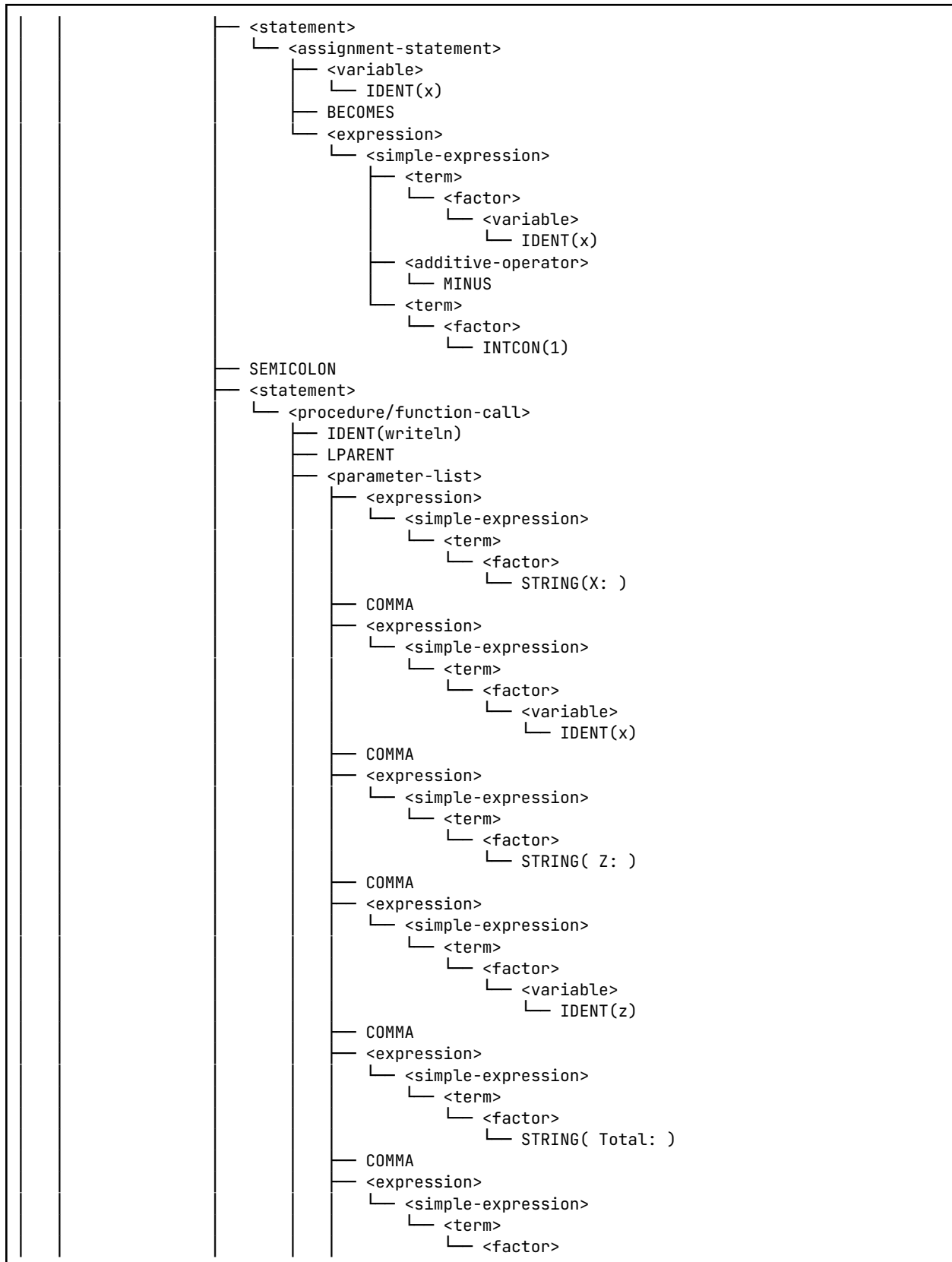
```

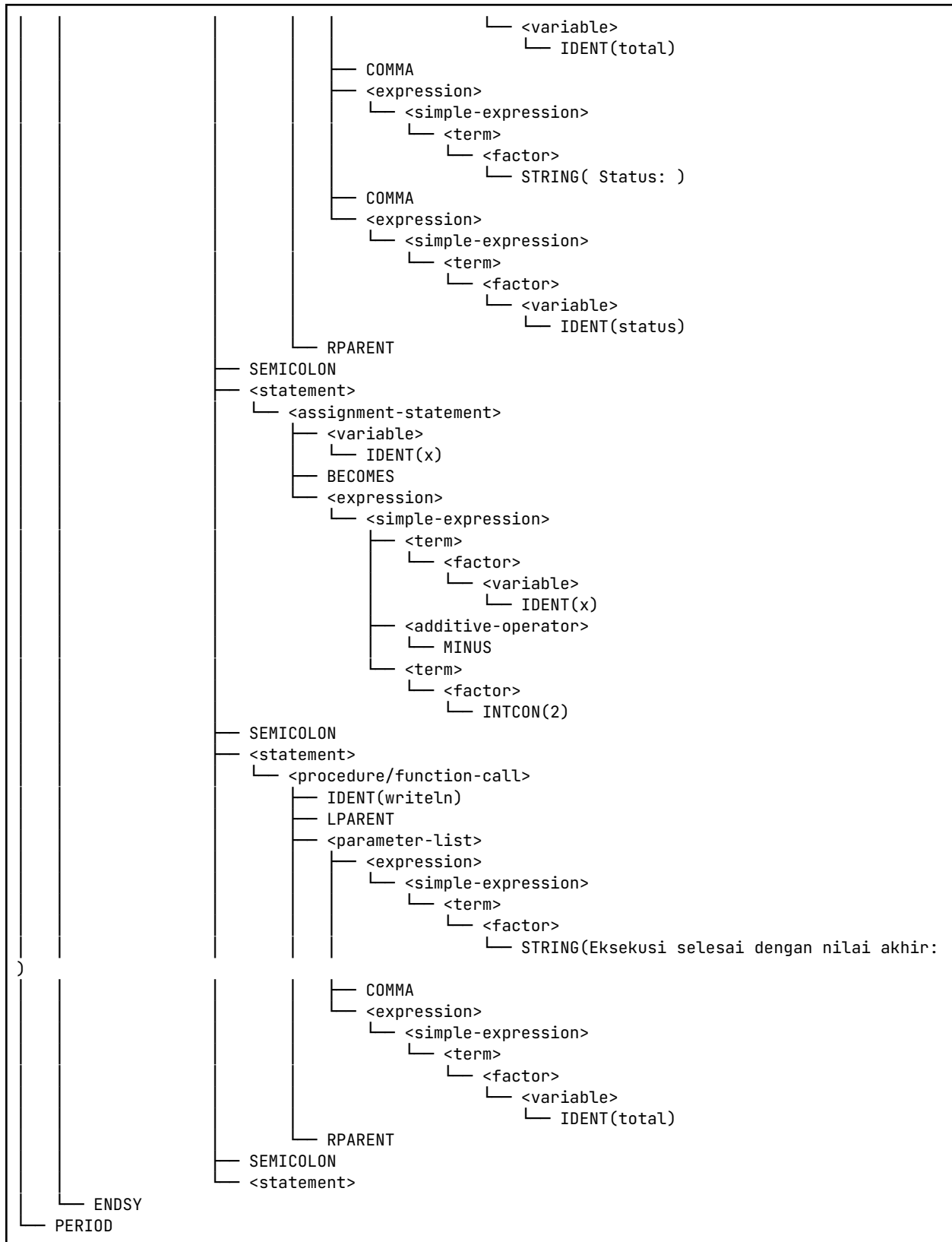
— <statement>
  — <assignment-statement>
    — <variable>
      — IDENT(x)
    — BECOMES
    — <expression>
      — <simple-expression>
        — <term>
          — <factor>
            — INTCON(10)
  — SEMICOLON
— <statement>
  — <assignment-statement>
    — <variable>
      — IDENT(y)
    — BECOMES
    — <expression>
      — <simple-expression>
        — <term>
          — <factor>
            — INTCON(5)
  — SEMICOLON
— <statement>
  — <assignment-statement>
    — <variable>
      — IDENT(z)
    — BECOMES
    — <expression>
      — <simple-expression>
        — <term>
          — <factor>
            — INTCON(0)
  — SEMICOLON
— <statement>
  — <assignment-statement>
    — <variable>
      — IDENT(total)
    — BECOMES
    — <expression>
      — <simple-expression>
        — <term>
          — <factor>
            — INTCON(0)
  — SEMICOLON
— <statement>
  — <assignment-statement>
    — <variable>
      — IDENT(status)
    — BECOMES
    — <expression>
      — <simple-expression>
        — <term>
          — <factor>
            — CHARCON(N)
  — SEMICOLON
— <statement>
  — <while-statement>
    — WHILESY

```









5. Test Case 5

input5.txt

```
program DataProcessing;
var
  a, b, limit: integer;
  ArrayA, ArrayB: array [1..5] of integer;
  Kode: array [1..3] of char;
begin
  a := 1;
  b := 5;
  limit := 5;

  Kode[1] := 'A';
  Kode[2] := 'B';
  Kode[3] := 'C';

  while a < limit + 1 do
    ArrayA[a] := a * 2;
    ArrayB[a] := b * 2;

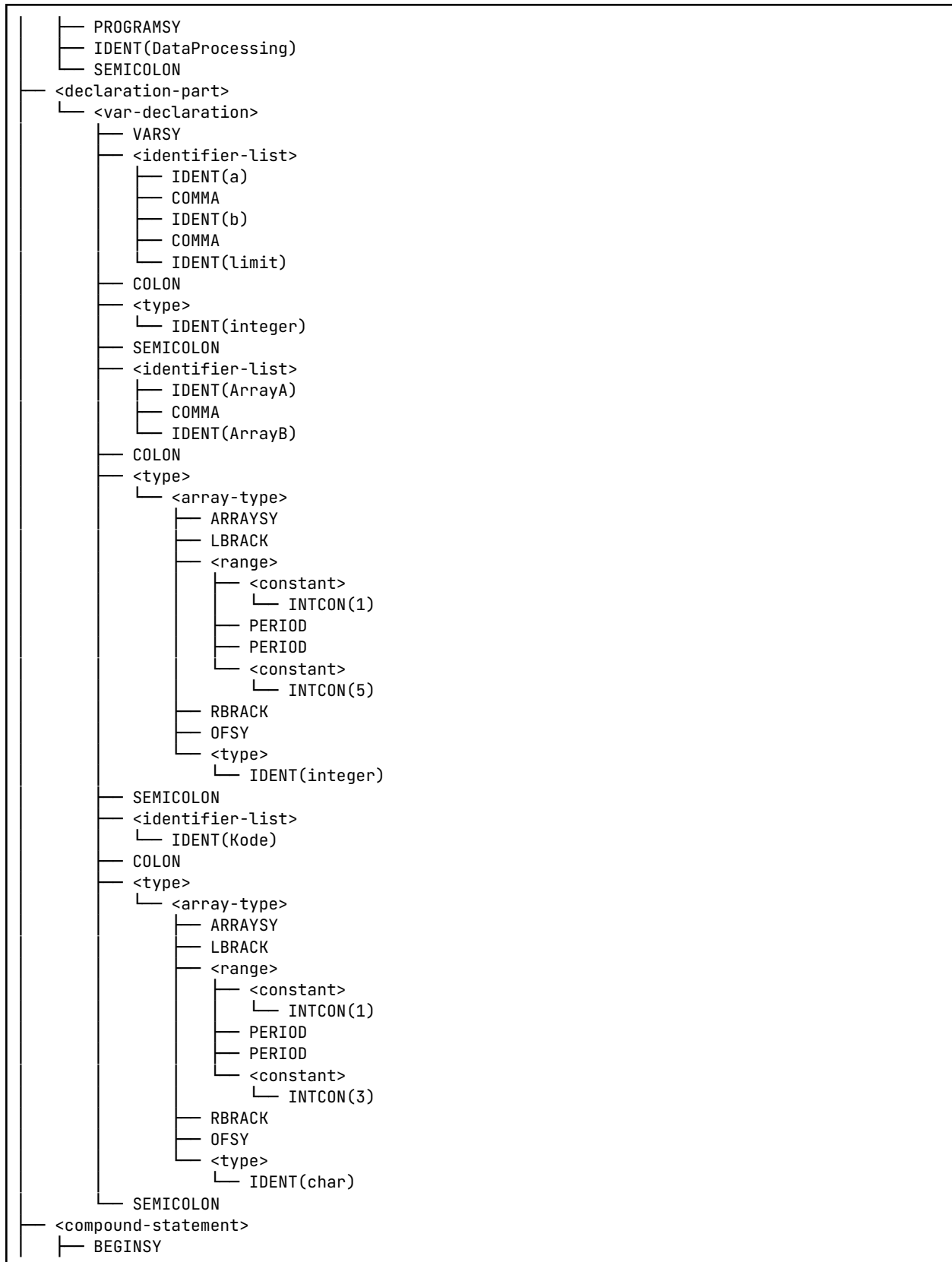
    if (ArrayA[a] > ArrayB[a]) then
      writeln('ArrayA menang dengan nilai ', ArrayA[a])
    else
      writeln('ArrayB menang dengan nilai ', ArrayB[a]);

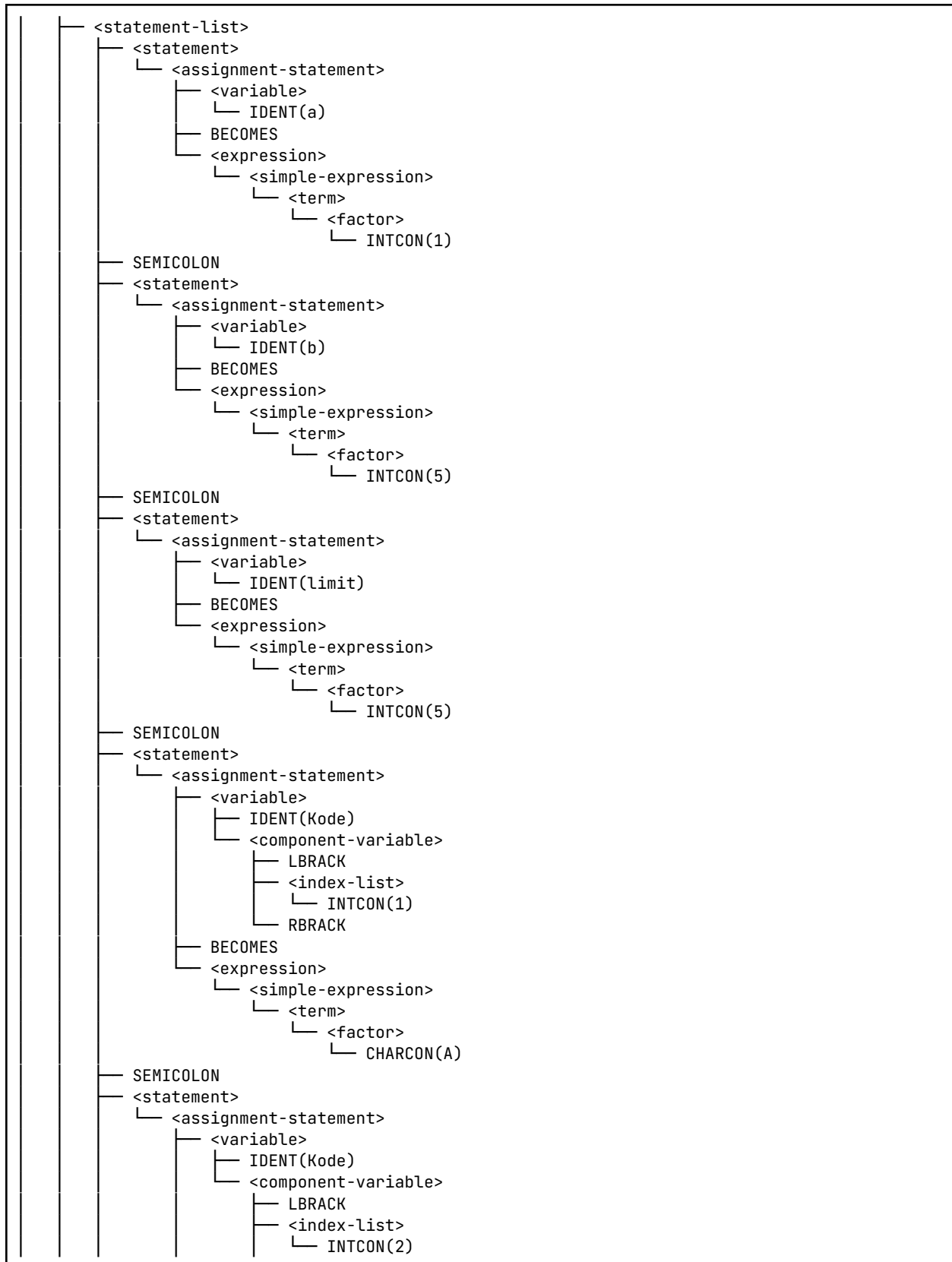
    a := a + 1;
    b := b - 1;

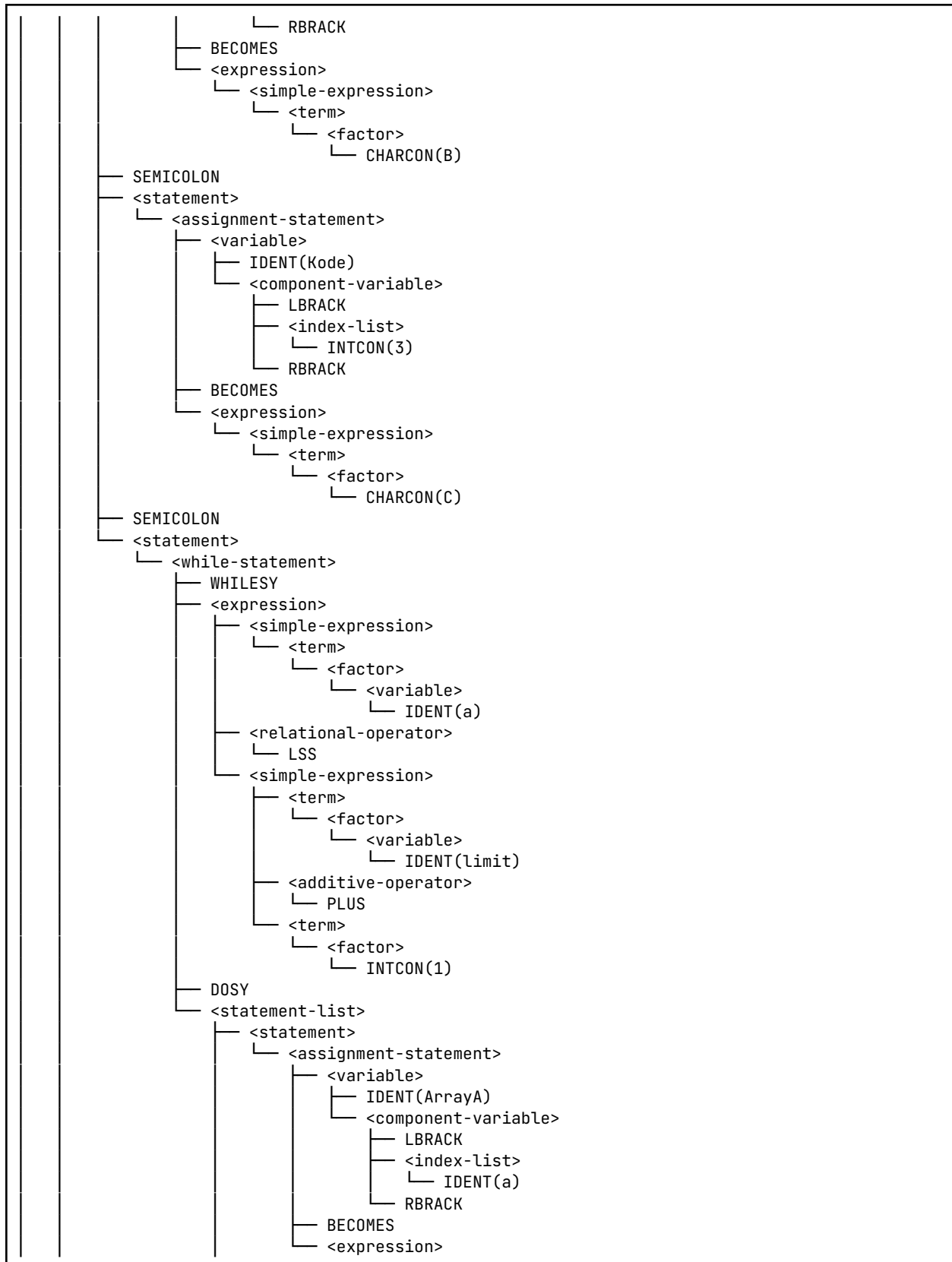
  writeln('Kode eksekusi terakhir: ', Kode[3]);
end.
```

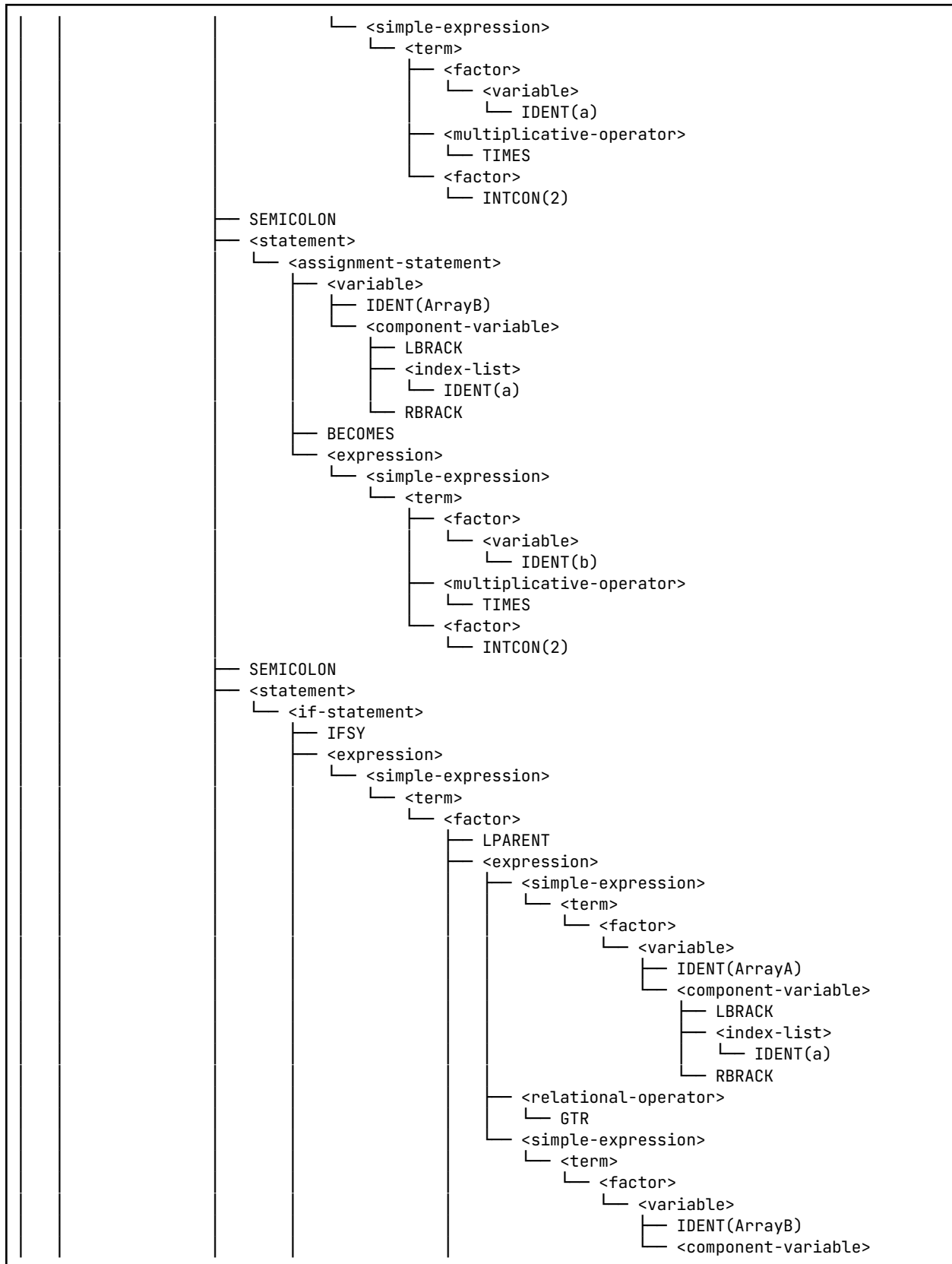
output5.txt

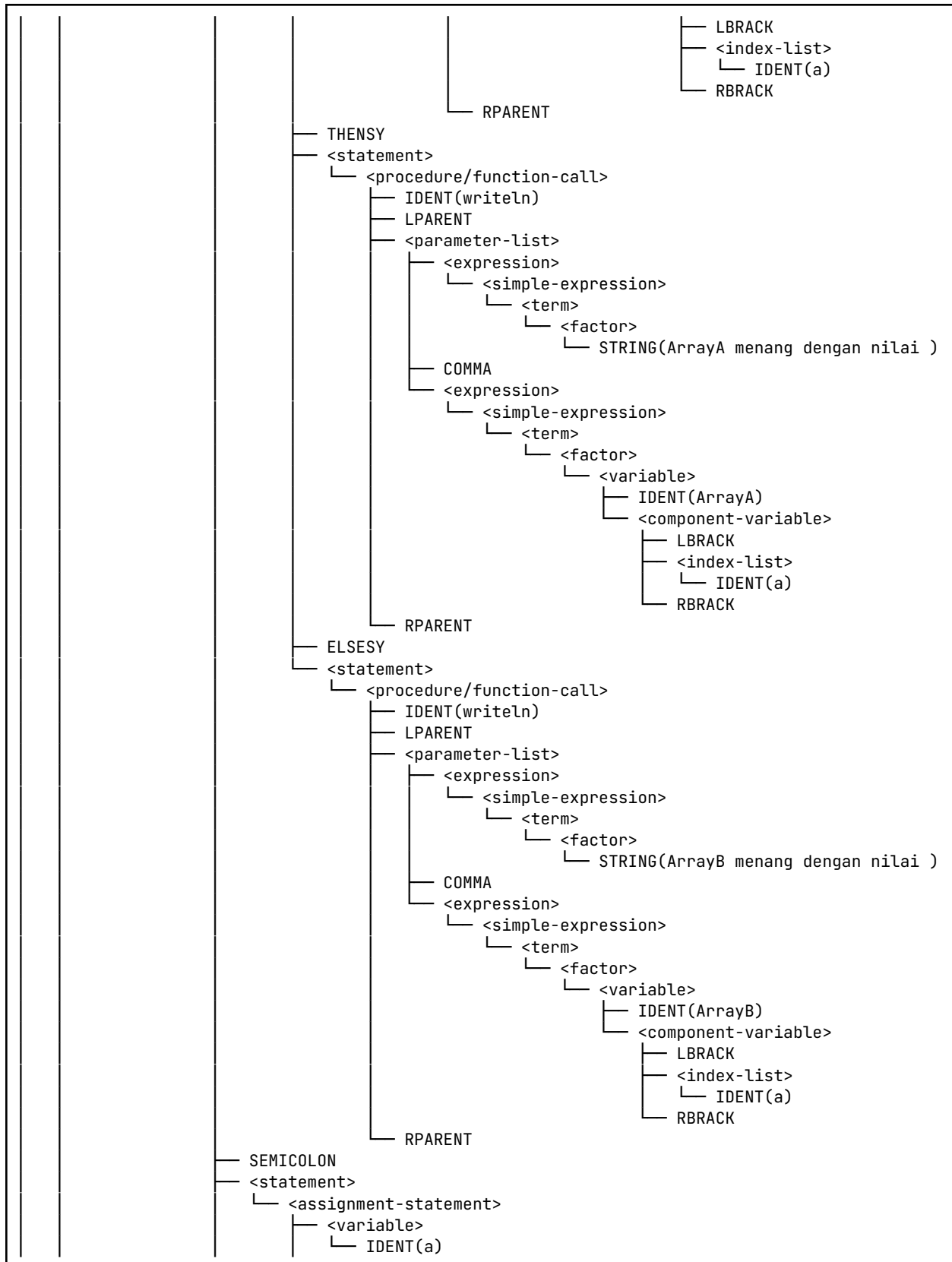
```
<program>
└─ <program-header>
```











BAB 4

KESIMPULAN DAN SARAN

1. Kesimpulan

Pada Milestone 2 ini, telah berhasil diimplementasikan sebuah Syntax Analyzer (Parser) untuk bahasa pemrograman Arion menggunakan algoritma Recursive Descent. Parser dibangun di atas fondasi Lexer dari Milestone 1, membentuk pipeline yang kohesif dari source code mentah hingga representasi Parse Tree yang terstruktur dan dapat dibaca.

Parser yang dihasilkan mampu menangani keseluruhan grammar bahasa Arion yang mencakup lebih dari 40 aturan produksi, mulai dari struktur program paling atas (<program>) hingga unit ekspresi terkecil (<factor>). Seluruh konstruksi bahasa utama telah diimplementasikan: deklarasi konstanta, tipe data (termasuk array, record, dan enumerated), variabel, prosedur dan fungsi beserta parameter formal, akses array dan field record melalui <component-variable>, serta seluruh jenis statement yaitu if-else, case, while, repeat-until, dan for dengan arah iterasi to maupun downto.

Arsitektur modular yang memisahkan implementasi ke dalam empat file .cpp berdasarkan tanggung jawab terbukti efektif untuk pengembangan paralel dalam tim. Setiap anggota dapat mengerjakan modulnya masing-masing tanpa konflik, dengan Parser.hpp sebagai kontrak interface bersama. Mekanisme error handling memastikan syntax error tidak menyebabkan crash, dengan pesan yang menyebutkan jenis token bermasalah dan posisinya secara tepat. Parse Tree yang dihasilkan dicetak dengan format visual menggunakan karakter box-drawing UTF-8 yang memudahkan verifikasi kebenaran struktur secara manual.

2. Saran

Pertama, mekanisme error recovery masih terbatas. Setelah error pertama ditemukan, flag `hasError` menghentikan pelaporan error berikutnya. Implementasi panic-mode error recovery, di mana parser melewati token-token hingga menemukan token sinkronisasi seperti SEMICOLON atau END, akan memungkinkan pelaporan lebih dari satu error dalam satu kali eksekusi, yang jauh lebih berguna bagi pengguna.

Kedua, `parseStatement()` saat ini menggunakan lookahead sederhana untuk membedakan assignment dari procedure call. Untuk kasus seperti `arr[i] := 5`, lookahead token kedua adalah LBRACK yang sudah ditangani, namun kasus berantai seperti `rec.field.subfield := x` mungkin perlu perhatian lebih dalam implementasi `parseComponentVariable()` pada milestone semantic.

Ketiga, spesifikasi grammar mengalami 5 kali revisi selama masa pengerjaan Milestone 2, yang menyebabkan penyesuaian berulang pada implementasi dan laporan. Untuk milestone berikutnya, sangat disarankan agar tim meninjau dokumen spesifikasi secara kolektif di awal pengerjaan dan menetapkan satu sumber kebenaran grammar yang disepakati bersama sebelum pembagian tugas dilakukan.

LAMPIRAN

Tautan repository Github:

<https://github.com/Rizelbit/SBT-Tubes-IF2224-2026>

Tabel Pembagian Tugas

No.	NIM	Persentase Pembagian Tugas (%)
1.	13524032	25
2.	13524036	25
3.	13524056	25
4.	13524102	25

REFERENSI

“Introduction to Automata Theory, Languages, and Computation 3rd Edition”

https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

“Compilers - Principles, Techniques, and Tools (2006)”

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)

“Introduction To Automata Theory Languages , and Computation - John Hopcroft.”

https://methodist.edu.in/web/uploads/files/Introduction%20To%20Automata%20Theory_john%20hopcroft.pdf